Reverse-KL Objectives in the Training, Fine-Tuning, and Reinforcement Learning of Language Models

A literature review, with an audit of what implementations actually optimize

August 2026 · sources verified against primary texts and commit-pinned code; full quote dossiers in `lit_review/notes/`

Contents

1 Introduction and conventions 1
2 The folklore and its foundations 2
2.1 The standard claims and the arguments behind them 2
2.2 The standard figures 6
2.2.1 Our own recreations 7
2.3 How LLM training papers motivate reverse KL 9
2.3.1 The folklore as the stated motivation 9
2.3.2 Tractability and operational control: the RLHF lineage 14
2.3.3 RL synergy, unhackability, density, and compute 15
2.3.4 Derived optimality 15
2.3.5 Estimator-level motivations 16
2.3.6 Adoption by precedent, without a direction argument 16
2.4 Criticisms and complications 17
2.4.1 KL-regularized RL specifies unimodal targets by design 18
2.4.2 The low-KL "agreement trap": local agreement without corrective signal 19
2.4.3 Degenerate agreement and student-induced teacher hacking 19
3 Reverse KL as a training objective in practice 20
3.1 Distillation research 20
3.2 RL and alignment 21
3.3 Production technical reports 23
3.4 Student-sampled objectives 26
4 Estimators and implementations 26
4.1 Taxonomy: which "reverse KL" is being estimated 26
4.2 Derivations: values versus gradients 27
4.3 Estimator audit table 31
4.4 Source table 34
4.5 Audit notes and derivations for the non-trivial rows 36
4.6 Support and decoding modifications that change which KL is estimated 37
5 Reported empirical effects 38
6 Reproducing the Tinker recipe 39
7 Criticism and alternatives 41
8 Synthesis 41
Bibliography 42

1 Introduction and conventions

Throughout this review, **reverse KL** (RKL) means

$$D_{\mathrm{KL}}(q \,\|\, p) = \mathbb{E}_{x\sim q}[\log q(x) - \log p(x)], \tag{1}$$

where q is always the distribution **being trained** (student or policy) and p the **target** (teacher or reference). **Forward KL** (FKL) is $D_{\mathrm{KL}}(p \,\|\, q)$. We flag every source whose terminology or letter assignment deviates. The review covers (i) the folklore on choosing between the two directions and its evidentiary basis (Section 2), (ii) where reverse KL actually appears in LM training — research methods, alignment pipelines, and production technical reports (Section 3), (iii) a code-level audit of what the associated implementations compute and which

gradients they take, with derivations (Section 4), (iv) reported empirical effects (Section 5), and (v) criticism and alternative divergences (Section 7), ending with a synthesis (Section 8).

Terminology flags. Non-trivially many sources deviate from the convention above; these deviations are a real hazard when reading this literature side by side.

Source	Deviation
PRML (Bishop, 2006)	Calls $D_{\mathrm{KL}}(p \parallel q)$ "the reverse Kullback-Leibler divergence" (p. 467) — the **opposite** of our convention, because his default direction is the variational-inference objective $D_{\mathrm{KL}}(q \parallel p)$.
BOND (Sessa *et al.*, 2024)	Eq. 11 uses p for the **trained policy** and q for the target — letters swapped relative to ours. Their "backward KL" $= D_{\mathrm{KL}}(\text{policy} \parallel \text{target})$ agrees with our reverse KL in direction; "backward" and "reverse" are used interchangeably.
vBoN (Amini *et al.*, 2024)	"Backward KL" = our reverse KL.
Ziegler *et al.* (2019); Korbak, Perez and Buckley (2022)	Comma notation "$\mathrm{KL}(\pi, \rho)$" for $D_{\mathrm{KL}}(\pi \parallel \rho)$; policy first, so direction matches ours.
Schulman (2020)	Writes $\mathrm{KL}[q, p]$ with the **first** argument the sampled distribution; with q the policy this is our reverse KL.
f-DPO (Wang *et al.*, 2024)	In f-divergence notation $D_f(p \parallel q) = \sum_x q f(p/q)$, their reverse KL is $f(u) = u \log u$ and forward KL is $f(u) = -\log u$ — the $\parallel$ order of the f-divergence does not visually match the KL subscript order.
Minitron (Muralidharan *et al.*, 2024)	Bare "KLD" means **forward** KL (explicit in their Table 15 caption: "forward KLD loss").
Minka (2005)	"Exclusive" divergences ($\alpha < 1$) = reverse-KL-like; "inclusive" = forward-KL-like. His α-parametrization ($\alpha = 0$ RKL, $\alpha = 1$ FKL) differs from Bishop's ($\alpha = -1$ RKL, $\alpha = +1$ FKL).
Tinker cookbook (code)	Docstring in `train_on_policy.py` writes "KL[p‖q] = log p − log q" with p denoting the **student's** sampled logprobs; the computed quantity is $\log q_{\text{student}} - \log p_{\text{teacher}}$, i.e. our reverse KL. Naming swapped, computation consistent.

2 The folklore and its foundations

2.1 The standard claims and the arguments behind them

The folklore, in one sentence: **forward KL is zero-avoiding / inclusive / mass-covering ("mean-seeking"); reverse KL is zero-forcing / exclusive / mode-seeking; use forward KL when coverage or likelihood matters, reverse KL when you want sharp, conservative, high-precision outputs and can tolerate dropped modes.** This section collects the canonical statements source by source — Minka and Bishop in detail; Murphy, MacKay, Goodfellow, and Huszár more briefly — asking three questions of each: what exactly is claimed, what argument (or source) supports it, and which assumptions are made explicit.

Minka (Minka, 2005) **(technical report; the primary source).** The claims are stated for the α-divergence family (Zhu–Rohwer parametrization: $\alpha = 0$ is reverse KL, $\alpha = 1$ is forward KL), first as the outcome of a worked example — a **global** minimization over a deliberately too-small family:

> The original distribution $p(x)$ is a mixture of two Gaussians, one tall and narrow, the other short and wide. The approximation $q(x)$ is required to be a single (scaled) Gaussian, with arbitrary mean, variance, and scale factor. For different values of α, figure 1 plots the global minimum of $D_\alpha(p \parallel q)$ over q. ... When α is a large negative number, the best approximation represents only one mode, the one with largest mass (not the mode which is highest). When α is a large positive number, the approximation tries to cover the entire distribution, eventually forming an upper bound when $\alpha \to \infty$.
>
> — (Minka, 2005) · *Divergence Measures and Message Passing*

(that figure is reproduced, with our recreation, in Figure 3). The argument for the general claim is a direct analysis of the objective — which term of the divergence formula penalizes what:

> The properties observed in this example are general, and can be derived from the formula for α-divergence. Start with the mode-seeking property for $\alpha \ll 0$. It happens because the valleys of p force the approximation downward. ... These divergences are zero-forcing because $p(x) = 0$ forces $q(x) = 0$. In other words, they avoid "false positives," to an increasing degree as α gets more negative. ... The cost of excluding an x, i.e. setting $q(x) = 0$, is $p(x)/(1 - \alpha)$. Therefore q will keep the areas of largest total mass, and exclude areas with small total mass. Zero-forcing emphasizes modeling the tails, rather than the bulk of the distribution, which tends to underestimate the variance of p.
>
> — (Minka, 2005) · *Divergence Measures and Message Passing*

> When $\alpha \geq 1$, a different tendency happens. These divergences want to cover as much of p as possible. Following the terminology of Frey et al. (2000), these divergences are inclusive ($\alpha < 1$ are exclusive). Inclusive divergences require $q > 0$ whenever $p > 0$, thus avoiding "false negatives."
>
> — (Minka, 2005) · *Divergence Measures and Message Passing*

The only external source in these passages is Frey et al. (2000) — for the **terminology**, not the behavior. Two nuances from the same section are rarely quoted downstream: the zero-forcing optimum keeps the **largest-mass** region rather than the highest peak (above), and its variance tracks a single component even when many overlap ("if p has 100 identical Gaussians in a row, forming a plateau, the optimal q is only as wide as one of them"). Most importantly, Minka's own use-recommendation — the "when to use which" half of the folklore — is explicitly **conditioned on capacity**, in his concluding remarks:

> If the approximating family is a good fit to the true distribution, then it doesn't matter which divergence measure you use, since all will give similar results. The only consideration at that point is computational complexity. If the approximating family is a poor fit to the true distribution, then you are probably safest to use an exclusive divergence, which only tries to model one mode. With an inclusive divergence, message passing probably won't converge at all. If the approximating family is a medium fit to the true distribution, then you need to consider the inference goal.
>
> — (Minka, 2005) · *Divergence Measures and Message Passing*

The exact-minimization assumption also surfaces explicitly, if in passing: his theorem on the mass of the optimal q "applies to an exact minimization over $\tilde{Z}$, which is generally not possible."

Bishop (Bishop, 2006) **(PRML §10.1; the textbook articulation).** The setting is variational inference, and both framing assumptions — an intractable target and a deliberately restricted family — are stated before any behavioral claim:

> [W]e shall suppose the model is such that working with the true posterior distribution is intractable. We therefore consider instead a restricted family of distributions $q(Z)$ and then seek the member of this family for which the KL divergence is minimized.
>
> — (Bishop, 2006) · *Pattern Recognition and Machine Learning*

The argument for zero-forcing is an analysis of the KL integrand:

> [T]here is a large positive contribution to the Kullback-Leibler divergence ... from regions of Z space in which $p(Z)$ is near zero unless $q(Z)$ is also close to zero. Thus minimizing this form of KL divergence leads to distributions $q(Z)$ that avoid regions in which $p(Z)$ is small. Conversely, the Kullback-Leibler divergence $\mathrm{KL}(p \parallel q)$ is minimized by distributions $q(Z)$ that are nonzero in regions where $p(Z)$ is nonzero.
>
> — (Bishop, 2006) · *Pattern Recognition and Machine Learning*

supported by two worked examples (both reproduced in Section 2.2). From the factorized fit to a correlated Gaussian (his Fig. 10.2) he draws the compactness claim:

> [T]he variance of $q(z)$ is controlled by the direction of smallest variance of $p(z)$ … It is a general result that a factorized variational approximation tends to give approximations to the posterior distribution that are too compact.
>
> — (Bishop, 2006) · *Pattern Recognition and Machine Learning*

From the single-Gaussian fit to a bimodal p (his Fig. 10.3) he draws the mode-locking claim and the folklore's practice recommendation, with the multimodality premise stated up front:

> In practical applications, the true posterior distribution will often be multimodal, with most of the posterior mass concentrated in some number of relatively small regions of parameter space. … [A] variational treatment based on the minimization of $\mathrm{KL}(q \parallel p)$ will tend to find one of these modes. By contrast, if we were to minimize $\mathrm{KL}(p \parallel q)$, the resulting approximations would average across all of the modes and, in the context of the mixture model, would lead to poor predictive distributions (because the average of two good parameter values is typically itself not a good parameter value).
>
> — (Bishop, 2006) · *Pattern Recognition and Machine Learning*

Note "will tend to find **one of** these modes": the reverse-KL solutions in his own figure are distinct local minima — panel (c) is captioned "[a]s in (b) but showing a different local minimum" — so which mode is found is an initialization accident (Figure 1). His α-divergence summary states the general claim with a hedge ("typically") doing quiet work:

> Then for $\alpha \leq -1$ the divergence is zero forcing, so that any values of x for which $p(x) = 0$ will have $q(x) = 0$, and typically $q(x)$ will under-estimate the support of $p(x)$ and will tend to seek the mode with the largest mass. Conversely for $\alpha \geq 1$ the divergence is zero-avoiding, … and typically $q(x)$ will stretch to cover all of $p(x)$, and will over-estimate the support of $p(x)$.
>
> — (Bishop, 2006) · *Pattern Recognition and Machine Learning*

No external source is cited for any of these claims; the argument is self-contained (the integrand analysis plus the worked examples). One terminology hazard for modern readers: Bishop reserves "the reverse Kullback-Leibler divergence" for $\mathrm{KL}(p \parallel q)$ — the **opposite** of current LLM usage — since his default direction is the variational one.

Murphy (Murphy, 2022; 2023) **(the modern textbook transmission).** PML1 (§6.2.6) fixes the now-standard vocabulary — forward KL "will be zero-avoiding or mode-covering, and will typically over-estimate the support of p", reverse KL "is called zero-forcing or mode-seeking behavior … will typically under-estimate the support of p" — and PML2 states the mechanism:

> q tends to cover both modes as it must be nonvanishing everywhere p is; this is called mode-covering or zero-avoiding behavior … to prevent $D_{\mathrm{KL}}(q \parallel p)$ from becoming infinite, we must have $q = 0$ whenever $p = 0$, which creates mode-seeking or zero-forcing behavior
>
> — (Murphy, 2023) · *Probabilistic Machine Learning: Advanced Topics*

The supporting argument is the most explicit in the canon: PML2 §5.1.4 derives both optima exactly in restricted families — the moment-matching optimum of forward KL for exponential-family q, and the precision-diagonal optimum of factorized reverse KL on a Gaussian target — the same derivations behind our recreations (Section 2.2.1). Murphy is also the only canonical source to attach a counter-example caveat to the compactness claim:

> Note, however, that minimizing the reverse KL does not always result in an overly compact approximation, as explained in [Tur+08].
>
> — (Murphy, 2023) · *Probabilistic Machine Learning: Advanced Topics*

where [Tur+08] is Turner, Berkes, Sahani, and MacKay's counter-examples report (Section 2.4).

MacKay (MacKay, 2003) **(ITILA Ch. 33; the tractability articulation).** The direction is chosen because it is the **evaluable** one: the variational free energy differs from $D_{\mathrm{KL}}(Q \parallel P)$ by a constant (his eq. 33.9), and "for a range of interesting energy functions, and for sufficiently simple approximating distributions, the variational free energy can be efficiently evaluated" — while the forward objective earns a bracketed warning:

> [Note that in general it is not possible to evaluate the objective function G, because integrals under the true distribution $P(x)$ are usually intractable.]
>
> — (MacKay, 2003) · *Information Theory, Inference, and Learning Algorithms*

The compactness claim is stated as an unqualified tendency — "[t]he approximations given by variational free energy minimization always tend to be more compact than the true distribution" — but is explored through exercises rather than proved (Ex. 33.5; Ex. 33.7 constructs a problem with **three** distinct reverse-KL minima), and his printed solution states the capacity assumption in the same breath as the claim:

> The best possible variational approximation is of course the target distribution P. Assuming that this is not possible, a good variational approximation is more compact than the true distribution. In contrast, a good sampler is more heavy tailed than the true distribution.
>
> — (MacKay, 2003) · *Information Theory, Inference, and Learning Algorithms*

Goodfellow (Goodfellow, 2017) **(GAN tutorial; folklore stated, then retracted).** The tutorial records the folklore as a belief:

> Previously, many people (including the author) believed that GANs produced sharp, realistic samples because they minimize the Jensen-Shannon divergence while VAEs produce blurry samples because they minimize the KL divergence between the data and the model.
>
> — (Goodfellow, 2017) · *NIPS 2016 Tutorial: Generative Adversarial Networks*

Its Fig. 14 (reproduced in Figure 5) restates the two-direction claim with the capacity assumption unusually explicit in the caption itself:

> The two directions of the KL divergence are not equivalent. The differences are most obvious when the model has too little capacity to fit the data distribution. … Because a single Gaussian cannot capture the true data distribution, the choice of divergence determines the tradeoff that the model makes.
>
> — (Goodfellow, 2017) · *NIPS 2016 Tutorial: Generative Adversarial Networks*

and the same section then withdraws the explanatory claim on empirical grounds:

> GANs often choose to generate from very few modes; fewer than the limitation imposed by the model capacity. The reverse KL prefers to generate from as many modes of the data distribution as the model is able to; it does not prefer fewer modes in general. This suggests that mode collapse is driven by a factor other than the choice of divergence.
>
> — (Goodfellow, 2017) · *NIPS 2016 Tutorial: Generative Adversarial Networks*

Huszár (Huszár, 2015) **(the generative-models articulation the LLM papers cite).** The claims are behavioral, aimed at sample quality rather than inference, and the reverse-KL claim carries its own support condition in the trailing clause:

> Minimising $\mathrm{KL}[P \parallel Q]$ often leads to models that overgeneralise, and sometimes produce samples that are very unlikely under P. … If we train a generative model by minimising $\mathrm{KL}[Q \parallel P]$, the model will very conservatively try to avoid any behaviour that is unlikely under P. This comes at the cost of ignoring modes

> of P completely, unless those additional modes can be modelled without introducing probability mass in regions where P has none.
>
> — (Huszár, 2015) · *How (not) to Train your Generative Model: Scheduled Sampling, Likelihood, Adversary?*

No citation is attached to these characterizations; the argument is a reading of which distribution weights the log-ratio in each objective, and the note's contribution is the interpolation knob ("minimising JS_π divergence for a range of $\pi \in (0, 1)$ allows us to interpolate between the behaviour of $\mathrm{KL}[P \parallel Q]$ and $\mathrm{KL}[Q \parallel P]$"; Fig. 1 reproduced in Figure 4). Theis, Oord and Bethge (2016) supplied the corresponding constructions for evaluation, concluding "log-likelihood and visual appearance of samples are therefore largely independent."

Taking stock. Across all six sources the argument has one shape: characterize the exact minimizer of each divergence over a family too small to contain the target. The two load-bearing assumptions are stated by the sources themselves, though rarely by those who cite them (Section 2.3): (i) **capacity limitation** — explicit in Minka's good-fit/poor-fit conditioning, Bishop's "restricted family", MacKay's "[a]ssuming that this is not possible", and Goodfellow's caption; and (ii) **exact global minimization** — Minka plots "the global minimum" and flags exact minimization as "generally not possible", while Bishop's own figure lands the reverse-KL fit in one of several local minima. Alongside the behavioral claims runs a separate, non-behavioral reason for the reverse direction: it is the **tractable** one — MacKay's bracketed note, Bishop's "working with the true posterior distribution is intractable", and the VI literature generally (Blei, Kucukelbir and McAuliffe, 2017) — an asymmetry that recurs unchanged in RLHF, where reverse KL is the only direction estimable from on-policy samples given log-prob access to p (Section 3.2).

2.2 The standard figures

The folklore is transmitted by a small set of recurring figures, worth knowing by name: Bishop PRML Fig. 10.2 (factorized q on a correlated Gaussian, both directions) and Fig. 10.3 (single Gaussian on a bimodal p: (a) forward covers, (b)/(c) reverse locks onto one of two local minima) (Bishop, 2006); Minka Fig. 1 (Gaussian fit across the α family from mode-matching to covering) (Minka, 2005); Murphy's redraws with public notebooks (PML1 Fig. 6.3; PML2 Figs. 5.1–5.2) (Murphy, 2022; 2023); Goodfellow Figs. 14–15 (Goodfellow, 2017); Theis et al. Fig. 1 (Theis, Oord and Bethge, 2016); Huszár Fig. 1 (generalized-JS sweep) (Huszár, 2015); and, in the LLM-distillation literature, MiniLLM Fig. 2, which fits "a Gaussian mixture with a Gaussian distribution using forward KLD and reverse KLD" (Gu *et al.*, 2024), and GKD Fig. A.16 (its caption is quoted in Section 2.3) (Agarwal *et al.*, 2024). Figures 1–4 show the originals, with our numerical recreations collected in Section 2.2.1; Figure 5 shows the GAN-era redrawing, and each LLM paper's own folklore figure is reproduced beside its entry in Section 2.3. The originals are reproduced from their sources for private study and commentary in this unpublished working document; the recreations are our own (generation script at `figs/make_figures.py`).

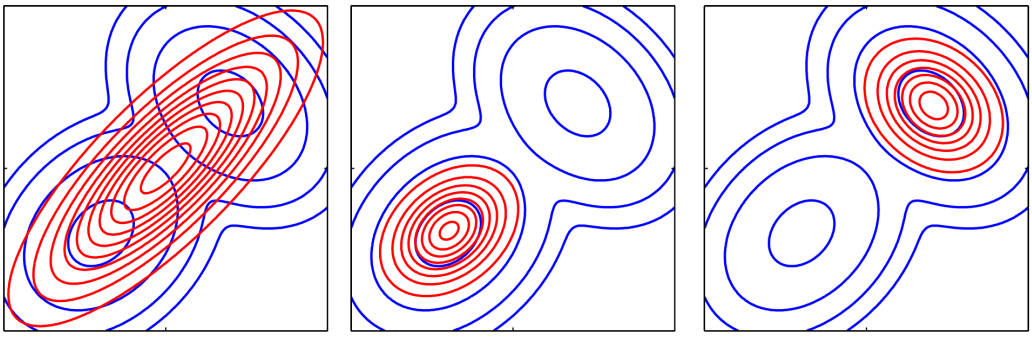


Figure 1: A single Gaussian fit to a bimodal p: the original Fig. 10.3, reproduced from PRML (Bishop, 2006) (p. 469, © 2006 Springer Science+Business Media): (a) the forward-KL fit covers both modes; (b)/(c) two distinct local minima of reverse KL, one per mode (blue $= p$, red $= q$). Our recreations: Figures 6 and 7.

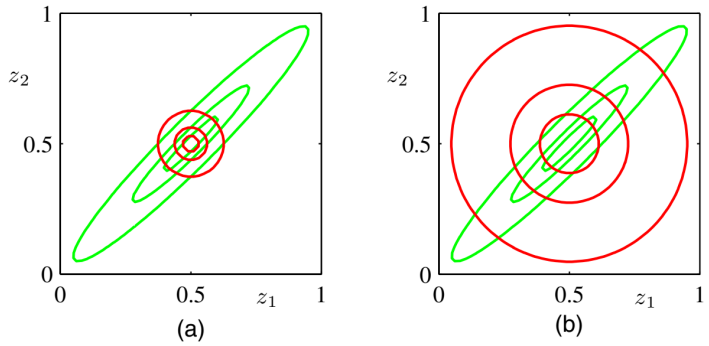


Figure 2: A factorized (diagonal-covariance) Gaussian q fit to a correlated Gaussian p: the original Fig. 10.2, reproduced from PRML (Bishop, 2006) (p. 468, © 2006 Springer Science+Business Media): green $= p$, red $= q$; their (a) minimizes $\mathrm{KL}(q \parallel p)$, their (b) minimizes $\mathrm{KL}(p \parallel q)$. Our recreation: Figure 8.

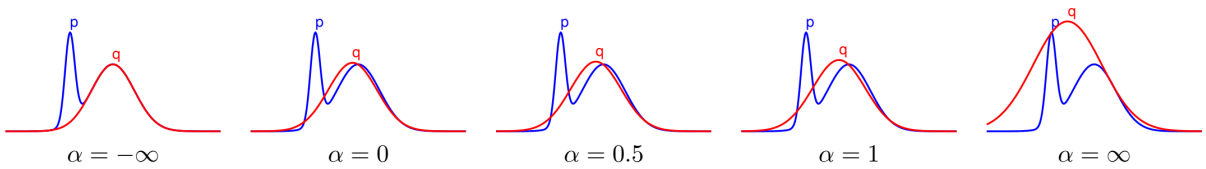


Figure 3: Gaussian fits minimizing the α-divergence, Zhu–Rohwer parametrization ($\alpha = 0$: reverse KL; $\alpha = 1$: forward KL): the original Fig. 1, reproduced from Minka's technical report (Minka, 2005) (© Microsoft Research); "$\alpha \to -\infty$ prefers matching one mode, while $\alpha \to \infty$ prefers covering the entire distribution." Our recreation: Figure 9.

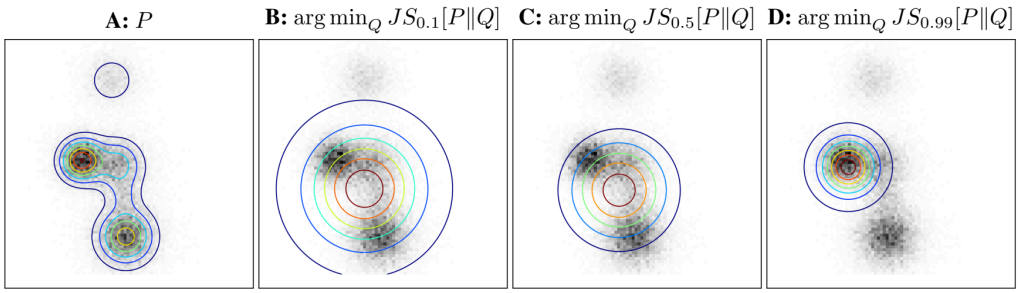


Figure 4: Gaussian fits minimizing Huszár's generalized JS_π: the original Fig. 1, reproduced from (Huszár, 2015) (arXiv:1511.05101): a 2-D mixture P (panel A) fit by a single isotropic Gaussian minimizing $\mathrm{JS}_\pi[P \parallel Q]$ for $\pi = 0.1, 0.5, 0.99$ (panels B–D). Our recreation: Figure 10.

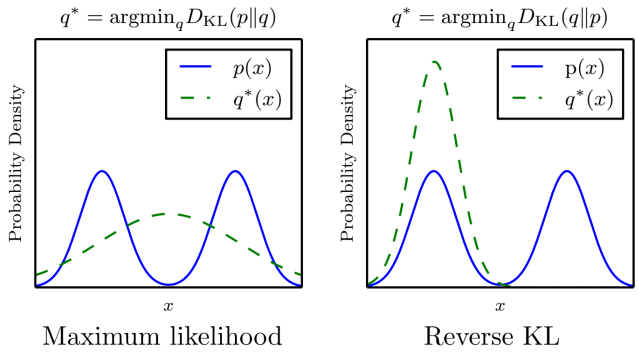


Figure 5: The same construction as redrawn in the GAN literature: Fig. 14, reproduced from the GAN tutorial (Goodfellow, 2017). The LLM-era redrawings appear beside their entries in Section 2.3.

2.2.1 Our own recreations

Our numerical recreations of the constructions in Figures 1–4, and of MiniLLM's toy figure (Figure 12).

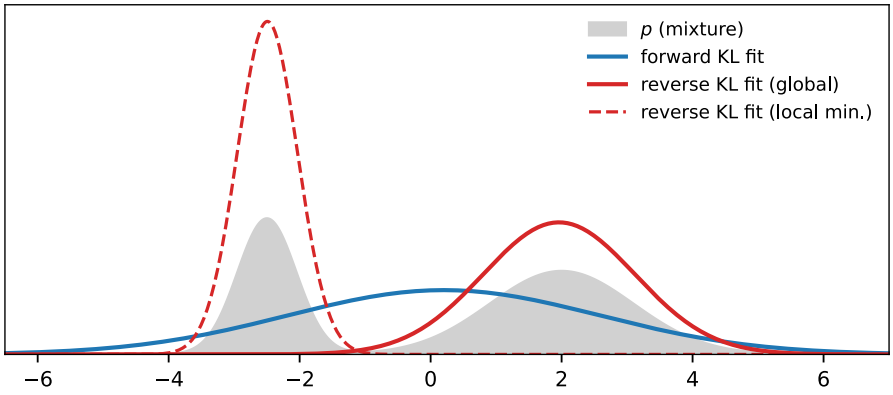


Figure 6: Our numerical recreation of Figure 1 on an asymmetric mixture. The forward-KL fit is the (exact) moment match and over-covers; the **global** reverse-KL minimum sits on the right-hand component, which carries the larger mass (0.6), **not** on the higher left-hand peak — the fit prefers "the one with largest mass (not the mode which is highest)" (Minka, 2005).

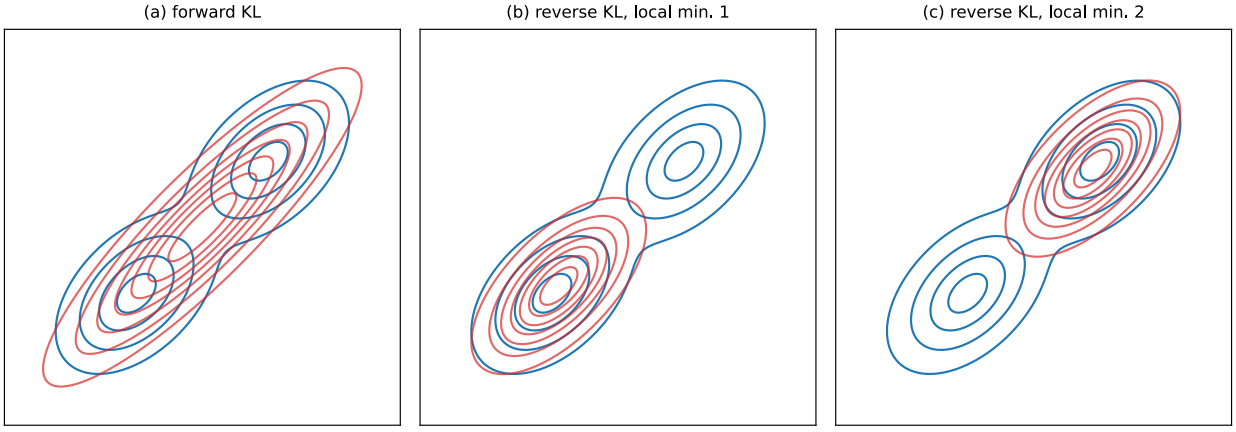


Figure 7: Our panel-by-panel recreation of Figure 1: a bimodal 2-D mixture p (blue) fit by a single full-covariance Gaussian q (red). (a) The forward-KL fit is the (exact) moment match and covers both modes; (b)/(c) the two local minima of reverse KL, each locked onto one mode.

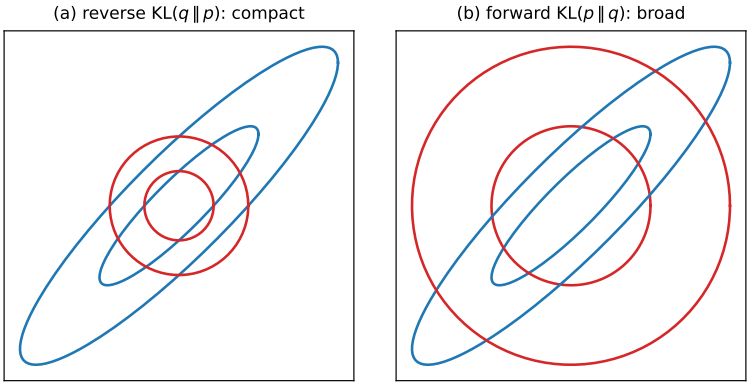


Figure 8: Our recreation of Figure 2 ($\rho = 0.9$; 1σ and 2σ contours; blue $= p$, red $= q$). The reverse-KL optimum matches the precision diagonal and is over-compact (variance $1 - \rho^2 = 0.19$); the forward-KL optimum matches moments and over-covers.

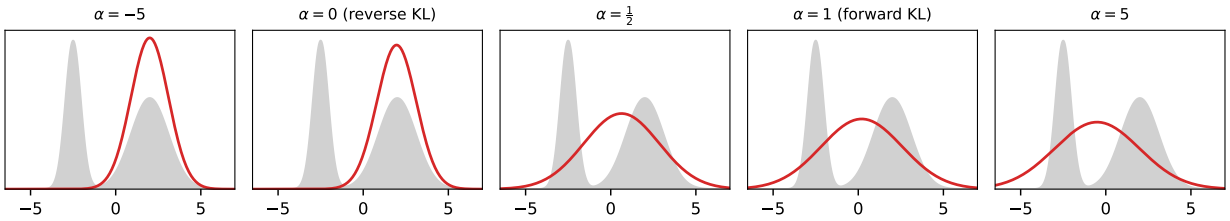


Figure 9: Our recreation of the α-divergence sweep in Figure 3, on our asymmetric mixture, with $\alpha = \pm 5$ standing in for $\alpha \to \pm\infty$. The family interpolates from mode-locking to mass-covering.

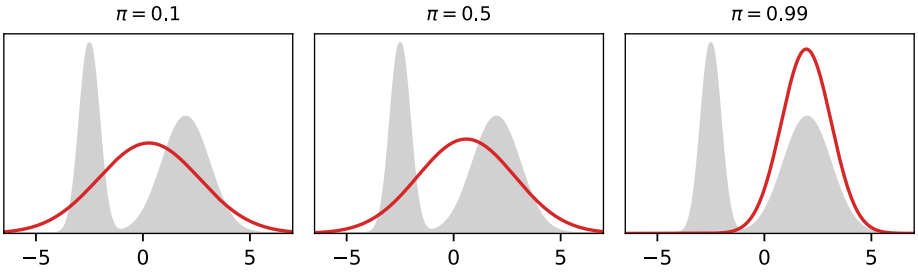


Figure 10: Our 1-D recreation of the JS_π sweep in Figure 4. As $\pi \to 0$ the objective behaves like (rescaled) forward KL; as $\pi \to 1$, like reverse KL — the interpolation Huszár proposed as the "ideal perceptual objective" knob.

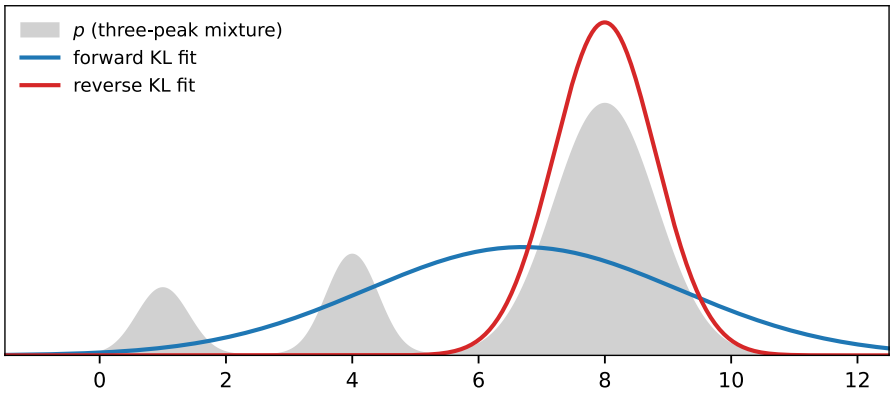


Figure 11: Our recreation of MiniLLM's own folklore figure (Figure 12): a three-peak Gaussian mixture (weights $0.1, 0.15, 0.75$) fit with a single Gaussian. The forward-KL fit is the (exact) moment match and covers all three peaks; the reverse-KL fit locks onto the dominant right-hand mode.

2.3 How LLM training papers motivate reverse KL

Which papers appear here is itself a methodological choice. An earlier draft of this section collected papers **because** they invoke the mode-seeking folklore — a selection rule that cannot show how reverse-KL adopters actually motivate the choice, since it filters out every adopter who reasons differently, or not at all. The entries below are instead selected on **usage**: the papers and production reports of Section 3 that adopt reverse KL, argue about its direction, or reject it, grouped by the motivation they actually give. Two caveats remain: silent adopters leave nothing to quote (they are listed explicitly in Section 2.3.6), and any quote collection still finds folklore vocabulary more easily than silence.

2.3.1 The folklore as the stated motivation

The folklore is the dominant stated motivation in the 2023–24 research literature — sometimes with the textbook argument spelled out, more often by citation chain or the bare words "mode-seeking". Each entry reproduces the exact citations the paper attaches to the claim; where a paper draws its own folklore figure, we reproduce it beside the entry. Notably, none of these papers restates the capacity-limitation or global-optimality assumptions on which the original argument rests (Section 2.1) when invoking the folklore; the closest approach is PromptKD's ablation conjecture, which rediscovers the capacity condition empirically rather than citing it.

MiniLLM (Gu *et al.*, 2024) **(distillation).** The introduction motivates the switch to reverse KL by indicting the forward direction:

> Minimizing forward KLD causes q_θ to assign unreasonably high probabilities to the void regions of p [MG19] and produces very unlikely samples under p during free-run generation [Hus15].
>
> — (Gu *et al.*, 2024) · *MiniLLM: Knowledge Distillation of Large Language Models*

and the method section states the reverse direction's virtue with a four-citation cluster:

> Minimizing reverse KLD has been shown to cause the mode-seeking behavior in generative modeling [Hus15, NCT16, CDP+18, LPSK23], where q_θ assigns high probabilities to p's large modes and ignore the small ones.
>
> — (Gu *et al.*, 2024) · *MiniLLM: Knowledge Distillation of Large Language Models*

In MiniLLM's bibliography, [Hus15] is Huszár's note (Huszár, 2015), [NCT16] is f-GAN (Nowozin, Cseke and Tomioka, 2016), [MG19] is Malinin and Gales's reverse-KL training of prior networks (Malinin and Gales, 2019), and [CDP+18] and [LPSK23] are a symmetric-VAE paper (Chen et al., 2018) and a vision self-distillation paper (Lee et al., 2023); the introduction's version of the void-regions claim additionally cites [M+05] — Minka's report, listed there as "Tom Minka et al. … Technical report, Citeseer, 2005". MiniLLM also draws the folklore itself (Figure 12).

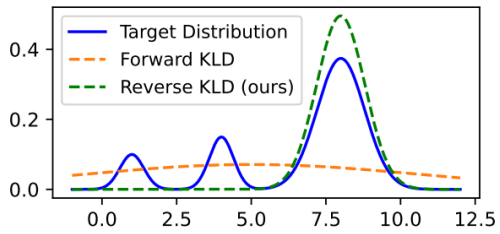


Figure 12: MiniLLM's own folklore figure (its Fig. 2): a three-peak Gaussian mixture fit with a single Gaussian under forward and reverse KLD. Reproduced from (Gu *et al.*, 2024). Our recreation: Figure 11.

GKD (Agarwal *et al.*, 2024) **(distillation).** The background section states the claim in full, citing only the paper's own appendix figure:

> Given model capacity mismatch, when approximating $P(C)$ using a distribution $Q_\theta(C)$, minimizing the reverse and forward KL results in mean and mode-seeking behavior (Figure A.16).
>
> — (Agarwal *et al.*, 2024) · *On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes*

That figure's caption is the textbook claim verbatim:

> [Reverse KL] is mode-seeking as it forces Q_θ to be zero where P is zero and hence makes it concentrate on one of the modes.
>
> — (Agarwal *et al.*, 2024) · *On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes*

with the trade-off stated in the body text:

> [Mode-seeking divergences] prioritize the tokens where the teacher assigns high probability, which can avoid low-quality generations but at the expense of less diverse generations for a given input.
>
> — (Agarwal *et al.*, 2024) · *On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes*

None of these statements carries a citation; the sole source anywhere in the chain is the caption's closing "See Le (2017) to replicate this plot", which resolves in GKD's bibliography to a personal blog note (Le, 2017). Fig. A.16 is reproduced in Figure 13.

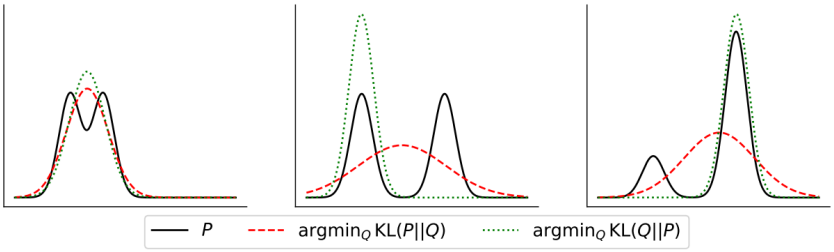


Figure 13: GKD's own folklore figure (its Fig. A.16), whose caption is quoted above. Reproduced from (Agarwal *et al.*, 2024).

PromptKD (Kim, Jang and Yang, 2024) **(distillation).** The third member of the "recent methods" trio the critics name below, it gives the direction exactly one sentence:

> Regarding the two objectives, we opt for minimizing the reverse KL divergence instead of the forward KL divergence to measure the discrepancy, as it exhibits mode-seeking behavior (Nowozin et al. 2016) and benefits generation tasks.
>
> — (Kim, Jang and Yang, 2024) · *PromptKD: Distilling Student-Friendly Knowledge for Generative Language Models via Prompt Tuning*

The sole citation is f-GAN (Nowozin, Cseke and Tomioka, 2016). The two axes are motivated separately, GKD-style: the student-generated pseudo-targets rest on "the insight (Agarwal et al. 2024) that incorporating sentences that the model can generate during free-run generation into the training process can mitigate exposure bias" — a data argument — while the sentence above carries the direction for the two prompt-tuning objectives; the student loss (reverse KL as well) gets no argument of its own. It draws no folklore figure, and its own appendix ablation — swapping each prompt-tuning loss to forward KL — deflates the choice:

> [E]xperimental results indicate that using reverse KL divergence yields the best performance. However, there is barely any significant difference. We conjecture that since the model distribution being trained is derived from the teacher, resulting in similar or even more modes in distribution, which prevent undesirable behaviors such as mode-covering even during forward KL divergence minimization.
>
> — (Kim, Jang and Yang, 2024) · *PromptKD: Distilling Student-Friendly Knowledge for Generative Language Models via Prompt Tuning*

The conjecture is Minka's capacity conditioning rediscovered empirically (Section 2.1): the trained distribution here is the prompted **teacher**, which has capacity to spare — so the direction stops mattering. (The student-loss direction is not ablated.)

f-DISTILL (Wen *et al.*, 2023) **(distillation).** It names the failure mode directly, citing no source for it:

> RKL distillation has the opposite problem, known as mode collapsing, where the student only learns one or a few modes of the teacher distribution. This is because the RKL loss would be large, if $q_\theta(Y')$ is high but $p(Y')$ is low for some Y'.
>
> — (Wen *et al.*, 2023) · *f-Divergence Minimization for Sequence-Level Knowledge Distillation*

Its Fig. 1 draws the folklore as "typical phenomena" (Figure 14).

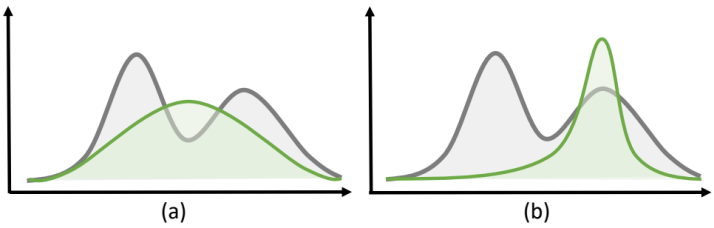


Figure 14: f-DISTILL's own folklore figure (its Fig. 1): "Typical phenomena of (a) KL distillation and (b) reverse KL distillation. Gray curve: teacher distribution. Green curve: student distribution." Reproduced from (Wen *et al.*, 2023).

PRML (Bishop, 2006) does appear in f-DISTILL's bibliography — but attached to the remark that cross-entropy training minimizes a KL, not to this claim.

DistiLLM-2 (Ko *et al.*, 2025) **(distillation).** It gives the mechanism-level version that motivates its direction–data pairing:

> KL increases $q_\theta(\cdot\,|x)$ in regions where $p(\cdot\,|x)$ are high (i.e., pulling-up effect) ... RKL attempts to reduce the ratio $q_\theta(\cdot\,|x)/p(\cdot\,|x)$, consequently $q_\theta(\cdot\,|x)$ decreases in region[s] where $p(\cdot\,|x)$ [is] small (i.e., pushing-down effect).
>
> — (Ko *et al.*, 2025) · *DistiLLM-2: A Contrastive Approach Boosts the Distillation of LLMs*

In the paper that passage opens "As shown in Figure 1, KL increases ...": the support is the paper's own toy figure, with no external source. Panel (a) of that figure — reproduced in Figure 15 — analyzes, per its caption, "long-tailed toy data introduced in Wu et al. (2024)": the toy data of the folklore's critics (Section 2.4), repurposed to argue the folklore's mechanism.

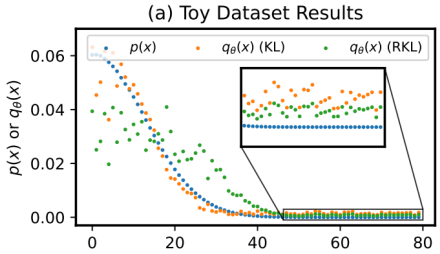


Figure 15: DistiLLM-2's own folklore figure (its Fig. 1(a)): KL and RKL "analyzed for long-tailed toy data introduced in Wu et al. (2024)". Reproduced from (Ko *et al.*, 2025).

Entropy-aware OPD (Jin *et al.*, 2026) **(distillation, 2026).** The 2026 wave states the folklore as its own finding:

> However, we show that the mode-seeking property of reverse KL reduces generation diversity and yields unstable learning signals when the teacher distribution has high entropy.
>
> — (Jin *et al.*, 2026) · *Entropy-Aware On-Policy Distillation of Language Models*

The abstract sentence carries no citation ("we show"); the body's statement of the claim does carry one:

> Since the expectation is taken over student samples, [the reverse KL objective] penalizes generated tokens that the teacher considers unlikely, but ignores teacher modes that the student does not visit. It induces *mode-seeking* behavior, encouraging the student to concentrate probability mass on a single high-likelihood mode of the teacher while ignoring others [29].
>
> — (Jin *et al.*, 2026) · *Entropy-Aware On-Policy Distillation of Language Models*

Its [29] is Minka's report (Minka, 2005) — making this one of the few invoking papers to reach the primary source.

AKL (Wu et al.) (Wu *et al.*, 2025) **(distillation; the critics).** They document the citation chain explicitly before attacking it:

> Therefore, recent methods (Gu et al., 2023; Agarwal et al., 2024; Kim et al., 2024) argue that RKL is more suitable than FKL to align the logits from LLMs since RKL avoids learning too many long-tail variants.
>
> — (Wu *et al.*, 2025) · *Rethinking Kullback-Leibler Divergence in Knowledge Distillation for Large Language Models*

Here Gu et al., 2023 is MiniLLM, Agarwal et al., 2024 is GKD, and Kim et al., 2024 is PromptKD (Kim, Jang and Yang, 2024) (above). Wu et al. are also the only invoking authors who return to the primary source — their analysis opens "Based on the analysis in Minka et al. (2005), reverse KL is zero forcing because $p = 0$ forces $q = 0$" — and their Fig. 1 (Figure 16) is captioned "The toy example in Gu et al. (2023)": MiniLLM's Gaussian figure, redrawn by the folklore's critics.

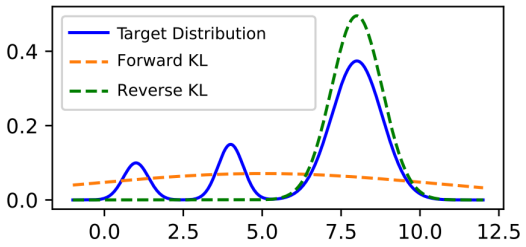


Figure 16: Wu et al.'s Fig. 1 — "The toy example in Gu et al. (2023)": MiniLLM's figure redrawn by the folklore's critics. Reproduced from (Wu *et al.*, 2025).

f-DPO (Wang *et al.*, 2024) **(alignment).** Motivating divergence constraints beyond reverse KL:

> The mode-seeking property of reverse KL divergence tends to reduce diversity in generation (Wiher et al., 2022; Khalifa et al., 2021; Perez et al., 2022; Glaese et al., 2022), which, although beneficial for optimizing alignment performance, may limit the model's potential (e.g., user engagement).
>
> — (Wang *et al.*, 2024) · *Beyond Reverse KL: Generalizing Direct Preference Optimization with Diverse Divergence Constraints*

Its four citations resolve to a study of decoding strategies (Wiher, Meister, and Cotterell, TACL 2022), GDC (Khalifa, Elsahar and Dymetman, 2021), red-teaming (Perez et al., EMNLP 2022), and Sparrow (Glaese et al., 2022) — alignment and decoding papers, not divergence theory. f-DPO also draws its own folklore figure (Figure 17).

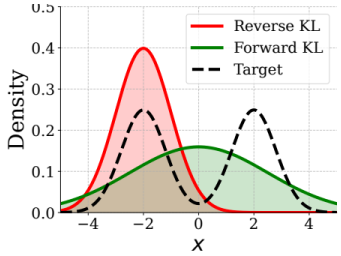


Figure 17: f-DPO's own folklore figure (its Fig. 1): "The mode seeking and mass covering behaviors of reverse KL and forward KL." Reproduced from (Wang *et al.*, 2024).

BOND (Sessa *et al.*, 2024) **(alignment).** It declares the folklore common knowledge while blaming both directions in one breath (recall BOND's swapped letters, Section 1: their p is the trained policy):

> ...the reverse KL$(p \parallel q)$ is well-known to have a mode-seeking effect, steering policy p to produce generations that have a high likelihood according to q (Agarwal et al., 2024). While the forward KL may produce over-spread distributions, the backward KL can lead to policy and entropy collapses.
>
> — (Sessa *et al.*, 2024) · *BOND: Aligning LLMs with Best-of-N Distillation*

The sole citation backing "well-known" is (Agarwal et al., 2024) — GKD again. BOND's figures are training-time KL curves, not folklore toys.

f-DPG (Go *et al.*, 2023) **(alignment).** It ties the LLM-era claim back to its generative-models lineage:

> ...low diversity but high quality for RKL-DPG, compared to other f-DPGs, suggesting it captures a subset of the target distribution ("mode collapse"), as commonly observed in other generative models (Huszar, 2015; Che et al., 2017; Mescheder et al., 2018).
>
> — (Go *et al.*, 2023) · *Aligning Language Models with Preferences through f-divergence Minimization*

Its lineage citations are Huszár (Huszár, 2015) and two GAN papers: mode-regularized GANs (Che et al., ICLR 2017) and a convergence analysis (Mescheder et al., ICML 2018).

Thinking Machines Blog (Lu and Thinking Machines Lab, 2025)

> [Reverse KL] is "mode seeking": it learns one specific behavior (the teacher's) instead of spreading its distribution across several suboptimal options.
>
> — (Lu and Thinking Machines Lab, 2025) · *On-Policy Distillation*

Minitron (Muralidharan *et al.*, 2024) **(technical report; secondhand).** It transmits the folklore as received wisdom while reporting the opposite result in its own regime:

> Recent work has shown R-KLD [15, 27] to be a better fit than KLD in the SFT/instruction-following setting, and Agarwal et al. [2] claim the choice of loss is task-dependent.
>
> — (Muralidharan *et al.*, 2024) · *Compact Language Models via Pruning and Knowledge Distillation*

— followed two sentences later by the finding that forward KL wins for pruned-base-model retraining (Section 3.3). In Minitron's numbered bibliography, [15] is MiniLLM (Gu *et al.*, 2024), [27] is DistiLLM (Ko *et al.*, 2024), and [2] is GKD (Agarwal *et al.*, 2024). It draws no folklore figure.

Traced one hop back, the citation chain is short and narrow. The technical report cites the LLM-distillation papers; the alignment papers cite GKD and empirical alignment work; the distillation papers cite the 2015–2018 generative-models literature (Huszár, f-GAN, GAN analyses — PromptKD's entire support is its lone f-GAN citation) or nothing at all (GKD's body text, f-DISTILL, DistiLLM-2); and the two figure-level invocations rest on blog posts (GKD's Fig. A.16 on Le's 2017 note (Le, 2017); TML's sidenote on Jang's 2016 tutorial (Jang, 2016)). The toy figure propagates like the claims: six of the invokers draw one (MiniLLM, GKD, f-DISTILL, DistiLLM-2, f-DPO, and Wu et al. — the last redrawing MiniLLM's, with DistiLLM-2 borrowing the critics' toy data). The textbook treatments are nearly absent: no invoking paper cites MacKay or Murphy; f-DISTILL lists PRML (Bishop, 2006) but attaches it elsewhere; and Minka is reached only three times — by MiniLLM's [M+05], by the critics (Wu *et al.*, 2025), and by entropy-aware OPD's [29] (Jin *et al.*, 2026).

2.3.2 Tractability and operational control: the RLHF lineage

The pipeline that made reverse KL ubiquitous (Section 3.2) never argues the direction at all: reverse KL is the only direction estimable from on-policy samples given log-prob access to the reference — the same tractability asymmetry that fixed the direction of variational inference (Section 2.1). The stated justifications are operational, and none of them cites a source:

> This term has several purposes: it plays the role of an entropy bonus, it prevents the policy from moving too far from the range where r is valid, and ... it also is an important part of the task definition.
>
> — (Ziegler *et al.*, 2019) · *Fine-Tuning Language Models from Human Preferences*

> First, it acts as an entropy bonus, encouraging the policy to explore and deterring it from collapsing to a single mode. Second, it ensures the policy doesn't learn to produce outputs that are too different from those that the reward model has seen during training.
>
> — (Stiennon *et al.*, 2020) · *Learning to Summarize from Human Feedback*

DPO inherits the same reverse-KL constraint in closed form and justifies it the same way:

> The added constraint is important, as it prevents the model from deviating too far from the distribution on which the reward model is accurate, as well as maintaining the generation diversity and preventing mode-collapse to single high-reward answers.
>
> — (Rafailov *et al.*, 2023) · *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*

Note the sign: relative to **unregularized** reward maximization, the same intuitions the folklore turns against reverse KL are invoked **for** it — the folklore supplies the vocabulary for both readings. f-DPO adds an algebraic motivation: among divergence constraints, reverse KL is the one whose partition function cancels in DPO-style derivations (Wang *et al.*, 2024).

2.3.3 RL synergy, unhackability, density, and compute

- **Thinking Machines blog** (Lu and Thinking Machines Lab, 2025) — the post's motivation portfolio leads not with the folklore but with RL structure and reward safety:

  > Reverse KL has natural synergy with RL, which generally optimizes a form of sequence-level reverse KL induced by the reward model. However, unlike most reward models in practice, the reverse KL is "unhackable" in the sense that low KL always corresponds to a high probability of desirable behavior from the teacher model's point of view.
  >
  > — (Lu and Thinking Machines Lab, 2025) · *On-Policy Distillation*

  with density and compute close behind ("distillation reaches the teacher's level of performance approximately 7-10x faster than RL"; Section 3.1). The folklore appears third in the list, as an argument **for** reverse KL:

  > [Reverse KL] is "mode seeking": it learns one specific behavior (the teacher's) instead of spreading its distribution across several suboptimal options.
  >
  > — (Lu and Thinking Machines Lab, 2025) · *On-Policy Distillation*

  The only source attached to "mode seeking" is a sidenote — "See Eric Jang's post for more discussion of mode seeking behaviors" — linking Jang's 2016 variational-inference tutorial (Jang, 2016); the neighbouring claim that reverse KL "reduces exposure bias" links Bengio et al.'s scheduled-sampling paper (arXiv:1506.03099). The post's graphics are method diagrams; it draws no folklore figure.

- **Qwen3** (Qwen Team, 2025) — efficiency and exploration, with no direction argument (the direction is not even stated; Section 3.3):

  > ...distillation achieves significantly better performance than reinforcement learning while requiring approximately only 1/10 of the GPU hours. Furthermore, distillation from teacher logits enables the student model to expand its exploration space and enhance its reasoning potential, as evidenced by the improved pass@64 scores.
  >
  > — (Qwen Team, 2025) · *Qwen3 Technical Report*

- **Nemotron-Cascade 2** (NVIDIA, 2026a) — operational stabilization:

  > [Multi-domain On-policy Distillation] serves as a critical stabilization point ... effective at recovering benchmark performance that may have regressed during earlier, more specialized stages.
  >
  > — (NVIDIA, 2026a) · *Nemotron-Cascade 2: Post-Training LLMs with Cascade RL and Multi-Domain On-Policy Distillation*

2.3.4 Derived optimality

The variational reading motivates the direction as a theorem rather than a preference: the KL-regularized RLHF objective **is** reverse-KL variational inference toward the tilted posterior (Korbak, Perez and Buckley, 2022), and

EXO selects reverse KL because it is "required by the alignment objective" — while arguing that DPO's own objective "corresponds to the forward KL" (Ji *et al.*, 2024) (both in Section 3.2).

2.3.5 Estimator-level motivations

Some adopters motivate not the divergence but the estimator built on it.

- **DistiLLM** — selects **skewed** reverse KL for bounded gradients and estimator stability, not mode behavior (Ko *et al.*, 2024) (Section 3.1).
- **Nemotron 3 Ultra** — keeps the divergence but selects the sampled-token estimator after full-distribution matching underperformed (Section 3.3) (NVIDIA, 2026b).
- **DeepSeek-V4** — argues the opposite side of the same estimator question:

> ...we adopt full-vocabulary logit distillation in our OPD. Preserving the complete logit distribution in calculating reverse KL loss yields more stable gradient estimates and ensures faithful distillation of the teachers' knowledge.
>
> — (DeepSeek-AI, 2026) · *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*

- **DeepSeek-V3.2** — keeps the RLHF penalty while correcting its k3 estimator (Section 4.2) (DeepSeek-AI, 2025).

2.3.6 Adoption by precedent, without a direction argument

By 2026 the frontier consolidation stages adopt reverse-KL on-policy distillation with no direction argument anywhere in the report: full-text search finds no mode-seeking or mass-covering language in DeepSeek-V4, Kimi K3, or GLM-5.

- **Kimi K3** — consolidation:

> We adopt Multi-Teacher On-Policy Distillation (MOPD) to consolidate these domain-specialized capabilities across varying reasoning efforts into a unified model [76, 136, 29].
>
> — (Kimi Team, 2026) · *Kimi K3: Open Frontier Intelligence*

  its precedents resolving to the Thinking Machines post [76], MiMo-V2-Flash [136], and DeepSeek-V4 [29].

- **GLM-5** — cross-stage recovery:

> To mitigate this issue, we perform on-policy cross-stage distillation as the final stage, adopting an on-policy distillation algorithm [14; 52; 51; 28] to swiftly recover the skills acquired in earlier SFT and RL stages (Reasoning RL and General RL).
>
> — (GLM Team, 2026) · *GLM-5: From Vibe Coding to Agentic Engineering*

  with precedents resolving to MiniLLM [14], Qwen3 [52], MiMo-V2-Flash [51], and the Thinking Machines post [28].

- **DeepSeek-V4** — states the objective bare:

> ...a single unified model is trained through on-policy distillation, wherein the unified model acts as the student learning to optimize the reverse KL loss with teacher models.
>
> — (DeepSeek-AI, 2026) · *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*

  The folklore chain of Section 2.3.1 has become a precedent chain: at the frontier, the direction is inherited from Qwen3, the Thinking Machines post, and MiniLLM the way an optimizer default is — and the argument, when one is given at all, is operational rather than distributional.

The folklore chain of Section 2.3.1 has become a precedent chain: at the frontier, the direction is inherited from Qwen3, the Thinking Machines post, and MiniLLM the way an optimizer default is — and the argument, when one is given at all, is operational rather than distributional.

2.4 Criticisms and complications

The folklore has accumulated serious qualifications, several from within its own canon.

Capacity. If q can represent p exactly, every f-divergence has the same global optimum; the direction distinction is a misspecification phenomenon. f-GAN states it for generative models:

> ...when the generative model is misspecified and does not contain the true distribution, the divergence function used for estimation has a strong influence on which model is learned.
>
> — (Nowozin, Cseke and Tomioka, 2016) · *f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization*

Wu *et al.* (2025) make the LLM-specific version precise — the classical picture assumes a Gaussian/continuous q, but a softmax head can represent any categorical distribution, so FKL and RKL share the optimum:

> Contrary to prior assertions that reverse Kullback-Leibler (RKL) divergence is mode-seeking and thus preferable over the mean-seeking forward Kullback-Leibler (FKL) divergence, this study empirically and theoretically demonstrates that neither mode-seeking nor mean-seeking properties manifest in KD for LLMs.
>
> — (Wu *et al.*, 2025) · *Rethinking Kullback-Leibler Divergence in Knowledge Distillation for Large Language Models*

The observable difference is transient — "FKL focuses on the head part and RKL focuses on the tail part at the beginning epochs". (Scope caveat: their theory concerns representable optima and their experiments use small students; it does not settle the capacity-limited regime the folklore invokes.)

Mode collapse is not (only) the divergence's fault. Goodfellow (2017), refuting the GAN version of the folklore that he had himself helped spread:

> GANs often choose to generate from very few modes; fewer than the limitation imposed by the model capacity. The reverse KL prefers to generate from as many modes of the data distribution as the model is able to; it does not prefer fewer modes in general. This suggests that mode collapse is driven by a factor other than the choice of divergence.
>
> — (Goodfellow, 2017) · *NIPS 2016 Tutorial: Generative Adversarial Networks*

Conversely, Arjovsky and Bottou (2017) show the popular $-\log D$ GAN loss is reverse-KL-dominated, with the sharpest theoretical statement of the asymmetry:

> ...this KL assigns an extremely high cost to generating fake looking samples, and an extremely low cost on mode dropping.
>
> — (Arjovsky and Bottou, 2017) · *Towards Principled Methods for Training Generative Adversarial Networks*

Optimization, not objective. Soletskyi, Gabrié and Loureiro (2024) show reverse-KL mode collapse in VI occurs even when "the variational family is statistically well-specified":

> ...mode collapse is not a by-product of insufficient model expressivity, but a consequence of the optimization procedure.
>
> — (Soletskyi, Gabrié and Loureiro, 2024) · *A Theoretical Perspective on Mode Collapse in Variational Inference*

Together with Bishop's multiple local minima and MacKay's exercise exhibiting three distinct reverse-KL minima (MacKay, 2003), this relocates much of the folklore from the objective to the dynamics.

The compactness "folk-theorem". Turner and Sahani (2011) call the VI compactness claim a folk-theorem outright:

> It is well known that variational approximations, like those derived in the vEM E-Step, tend to be compact (MacKay, 2003). In other words, the variational approximation has a tendency to have a smaller entropy than the true distribution. The evidence for this folk-theorem is reviewed in the next section.
>
> — (Turner and Sahani, 2011) · *Two Problems with Variational Expectation Maximisation for Time-Series Models*

They also cite counter-examples (Turner, Berkes, Sahani and MacKay, 2008); Murphy's PML2 carries the same caveat (Murphy, 2023). Li and Farnia (2023) show that under a rigorous definition of mode-seeking, Jensen-Shannon qualifies while "standard KL" — the forward/MLE direction in their GAN estimation setting — does not; the labels have to be re-derived per divergence rather than assumed. In RL, Chan *et al.* (2022) find the direction's practical stakes context-dependent: reverse KL carries stronger policy-improvement guarantees, forward KL more exploration, with often modest differences.

Folklore about LLM mode collapse specifically. The most-cited folklore source for "RLHF causes mode collapse", janus's *Mysteries of mode collapse* (janus, 2022), documents striking sharpening (token confidences "frequently above 99%") but carries an in-post correction that the model in question, `text-davinci-002`, "was not trained with RLHF" (it was supervised FeedME tuning) — the phenomenon is real, its original attribution was wrong, and no KL objective is implicated by that source. Rigorous evidence on RLHF and diversity is deferred to Section 5.

Three 2025–2026 papers sharpen the criticism inside the LLM training stack itself — the first at the level of the KL-regularized RL **objective**, the other two at the level of the on-policy-distillation **estimator**, where the failure mode is conditional (prefix-coupled) rather than modal.

2.4.1 KL-regularized RL specifies unimodal targets by design

- GX-Chen *et al.* (2025) show the mode-seeking/mass-covering labels do not transfer to KL-**regularized** reward maximization: both directions admit closed-form optimal-target families (their Eqs. 1–5),

$$J_\beta(\pi_\theta) = \mathbb{E}_{y\sim\pi_\theta}[R(y)] - \beta D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}), \qquad G_\beta(y) \propto \pi_{\mathrm{ref}}(y)\, e^{R(y)/\beta}; \tag{2}$$

$$J_{\mathrm{fwd}}(\pi_\theta) = \mathbb{E}_{y\sim\pi_\theta}[R(y)] - \beta D_{\mathrm{KL}}(\pi_{\mathrm{ref}} \,\|\, \pi_\theta), \qquad G_{\mathrm{fwd}}(y) = \frac{\beta\,\pi_{\mathrm{ref}}(y)}{\Lambda - R(y)}, \quad \Lambda > \max_y R(y), \tag{3}$$

so which modes survive is a property of the **target family** — set by the regularization strength β and the relative scales of rewards and reference probabilities — not of the divergence direction.
- Under the common configuration (equal verifiable rewards, small β), the specified target is essentially unimodal: "by construction, KL-regularized RL with equal rewards never promotes a low support answer" — making "diversity collapse a natural consequence of correctly solving the RL problem", with either KL direction.
- For this review: a **target-design** explanation of mode collapse in the regularizer role, complementing the capacity critique of (Wu *et al.*, 2025) and the optimization critique of (Soletskyi, Gabrié and Loureiro, 2024) — a third mechanism by which the folklore's picture misattributes the observed behavior to the divergence.
- Their fix, Mode Anchored Reward Augmentation (MARA), edits reward magnitudes ($\bar{r}_i = R(z) + \beta(\log \pi_{\mathrm{ref}}(z) - \log \pi_{\mathrm{ref}}(y_i))$ for high-reward samples) so the induced target spreads mass across all high-reward modes — again independent of KL direction.

> It is commonly believed that optimizing the reverse KL divergence results in 'mode seeking', while optimizing forward KL results in 'mass covering', with the latter being preferred if the goal is to sample from multiple diverse modes. We show – mathematically and empirically – that this intuition does not necessarily transfer well to doing reinforcement learning with reverse/forward KL regularization.
>
> — (GX-Chen *et al.*, 2025) · *KL-Regularized Reinforcement Learning is Designed to Mode Collapse*

> ...commonly used settings such as low regularization strength and equal verifiable rewards tend to specify unimodal target distributions, meaning the optimization objective is, by construction, non-diverse.
>
> — (GX-Chen *et al.*, 2025) · *KL-Regularized Reinforcement Learning is Designed to Mode Collapse*

2.4.2 The low-KL "agreement trap": local agreement without corrective signal

- Xin *et al.* (2026) study the standard on-policy distillation objective (their Eq. 2) — the token-level reverse KL on student rollouts, i.e. the frozen-visitation objective of Section 4.2,

$$\mathcal{L}_{\mathrm{OPD}}(\theta) = \mathbb{E}_{x,\, \hat{y}\sim\pi_\theta(\cdot|x)}\left[\sum_{t=1}^{T} D_{\mathrm{KL}}(\pi_\theta(\cdot\,|x,\hat{y}_{<t}) \,\|\, \pi_T(\cdot\,|x,\hat{y}_{<t}))\right]. \tag{4}$$

- The trap: once a rollout drifts into an unrecoverable prefix, the **conditional** teacher locally agrees with the degraded state — the per-token reverse KL is low precisely where supervision is most needed. More than 60% of rollouts exhibit at least one extended low-KL segment, and once entered, the divergence tends to stay below its pre-segment level.
- The pathology is a property of prefix-conditioning, with no analog in the folklore's unconditional fitting picture: per-prefix conditionals cannot veto a globally bad trajectory. Their remedy treats it as a **data** problem, not a divergence-direction problem: KAT monitors the sliding-window mean $z_t = \frac{1}{W}\sum_{i=t-W+1}^{t} d_i$ of per-token divergences (their Eq. 4) and terminates the rollout once enough consecutive windows fall below a calibrated threshold (their Eq. 7), improving avg@k by 2.66, pass@k by 3.43, and cutting rollout length by 59.7% on four math benchmarks.

> When the student drifts into an unrecoverable prefix, the teacher may locally agree with the degraded state, producing low reverse KL but little corrective training signal. … We identify this persistent regime as a low-KL agreement trap.
>
> — (Xin *et al.*, 2026) · *Escaping the KL Agreement Trap in On-Policy Distillation*

> Conditioned on this degraded state, the teacher's token-level distribution may carry little corrective information, instead producing continuations that are plausible under the erroneous prefix and therefore close to the student's distribution.
>
> — (Xin *et al.*, 2026) · *Escaping the KL Agreement Trap in On-Policy Distillation*

2.4.3 Degenerate agreement and student-induced teacher hacking

- Yu *et al.* (2026) analyze the same objective family (their Eq. 1, the sampled-token estimator of Section 4.3); their sharper claim is that the agreement signal is **gameable**: because the student controls the visited contexts, it can manufacture prefixes that bend the teacher's conditionals toward its own behavior.
- Concretely, repetition loops: repeated prefixes make the teacher increasingly favor the same continuation, driving the measured token-level KL to a median of 0.0004 nats on globally degenerate responses. This is the pathology MiniLLM's teacher-mixed sampling (Gu *et al.*, 2024) patched in 2023 — reward hacking via repetition (Section 3.1) — resurfacing at 2026 scale under the sampled-token estimator, now with a mechanism: conditional acceptance by the teacher of student-manufactured contexts.
- Their fix (TIDE) is asymmetric by token type: bounded Hellinger-shaped suppression of student-**excess** tokens and analytic teacher top-K cross-entropy injection for **deficit** tokens,

$$g_{\mathrm{exc}} = -\mathbb{E}\left[\sum_t m_t^{-}\,\mathrm{sg}[h(a_t)]\log q_\theta(o_t|s_t)\right], \quad h(a) = 2\big(e^{a/2}-1\big), \tag{5}$$

$$\mathcal{L}_{\mathrm{def}} = -\mathbb{E}\left[\sum_t m_t^{+}\sum_{v\in\mathcal{K}_t}\bar{p}_t(v)\log q_\theta(v|s_t)\right], \tag{6}$$

with $m_t^{\mp}$ masking excess/deficit positions and $\mathcal{K}_t$ the teacher top-K (their §3, Eq. 4). Under strong mismatch (Qwen3-8B → Qwen3-1.7B-Base): Avg@8 6.9% → 20.3%, response length 22,395 → 7,294 tokens, missing-boxed-answer rate 65.5% → 5.4%.

> …degenerate agreement, where students exploit repetitive loops to achieve near-perfect token agreement with the teacher despite globally flawed responses…

> — (Yu *et al.*, 2026) · *Mismatch Matters: On-Policy Distillation Beyond Token Agreement*

> ...the student can reduce the measured teacher–student divergence by generating contexts that make the teacher's outputs more aligned with its own...
>
> — (Yu *et al.*, 2026) · *Mismatch Matters: On-Policy Distillation Beyond Token Agreement*

3 Reverse KL as a training objective in practice

3.1 Distillation research

MiniLLM (Gu *et al.*, 2024) made sequence-level reverse KL the **primary** distillation objective: $\min_\theta \mathrm{KL}[q_\theta \parallel p]$ with responses sampled from the student, optimized by policy gradient. Its motivation restates the folklore (quoted in Section 2.3), plus a values claim:

> [G]enerating one correct response is sufficient ... for scenarios demanding high truthfulness and reliability.
>
> — (Gu *et al.*, 2024) · *MiniLLM: Knowledge Distillation of Large Language Models*

Three estimator-level patches (single-step decomposition, teacher-mixed sampling, length normalization) are needed to make the vanilla objective train at all; each patches a concrete pathology (variance accumulation, reward hacking via repetition, short-response bias) — evidence, before any code audit, that the naive sequence-level estimator is degenerate (Section 4). The paper itself concedes the divergence-level cost:

> Models optimized by minimizing reverse KLD are likely to lose modes, which affects generation diversity.
>
> — (Gu *et al.*, 2024) · *MiniLLM: Knowledge Distillation of Large Language Models*

(Citation hygiene: arXiv v6 of 2026 retitled the paper "MiniLLM: On-Policy Distillation of Large Language Models", colliding with GKD's title; cite by arXiv ID 2306.08543.)

GKD (Agarwal *et al.*, 2024) decoupled the two ingredients MiniLLM had fused: (i) **where prefixes come from** (a fraction λ of batches use student rollouts — motivated by "train-inference distribution mismatch", an argument about data, not divergence direction, inherited from ImitKD (Lin *et al.*, 2020)) and (ii) **which divergence** (generalized JSD(β) interpolating FKL at $\beta = 0$ to RKL at $\beta = 1$, computed full-vocabulary per position). Its empirical conclusion is deflationary for the direction question: the on-policy data axis gives the robust gains, while "the optimal divergence seems to be task-dependent" and "the choice of divergence is temperature-dependent" — mode-seeking helps under temperature sampling and hardly matters under greedy decoding.

PromptKD (Kim, Jang and Yang, 2024) carries the recipe into parameter-efficient form: reverse KL throughout, on student-generated pseudo-targets, with a soft prompt (0.0007% of the teacher's parameters) tuned so the teacher emits "student-friendly" distributions. Its own ablation is deflationary in the same way as GKD's: swapping directions makes "barely any significant difference" (quoted in Section 2.3).

DistiLLM (Ko *et al.*, 2024) attacks the estimator, not the direction: skewed divergences $D_{\mathrm{SKL}}^{(\alpha)} = \mathrm{KL}(p \parallel \alpha p + (1-\alpha)q)$ and $D_{\mathrm{SRKL}}^{(\alpha)} = \mathrm{KL}(q \parallel (1-\alpha)p + \alpha q)$ bound the gradient ratio and stabilize training, while an adaptive replay scheme avoids MiniLLM/GKD's cost of constant student generation ("up to 80%" of training time). SRKL is their best instruction-following loss. **DistiLLM-2** (Ko *et al.*, 2025) makes the direction–data pairing explicit: skewed **forward** KL on teacher-generated responses, skewed **reverse** KL on student-generated responses, on the pulling-up/pushing-down grounds quoted in Section 2.3; notably, interpolating the two losses on the **same** data "does not improve performance compared to using either SKL or SRKL alone." f-DISTILL (Wen *et al.*, 2023) had already found symmetric losses (JS, TVD) "consistently better than asymmetric ones (KL and RKL)" outside near-unimodal machine translation, with RKL showing the worst teacher coverage ("mode collapsing"). Speculative KD adds the complementary failure mode of pure on-policy data: early student samples are out-of-distribution for the teacher, "leading to inaccurate teacher feedback" (Xu *et al.*, 2025).

On-policy distillation as an RL substitute. The Thinking Machines blog (Lu and Thinking Machines Lab, 2025) popularized per-token reverse KL on student rollouts as a **dense, cheap, unhackable** replacement for RLVR:

> We sample trajectories from the student model and use a high-performing teacher to grade each token.
>
> — (Lu and Thinking Machines Lab, 2025) · *On-Policy Distillation*

Reported results: Qwen3-8B-Base reaching AIME'24 74.4% in 1,800 GPU-hours where RL took 17,920 GPU-hours for 67.6% — "distillation reaches the teacher's level of performance approximately 7-10x faster than RL" — plus a behavior-restoration experiment (recovering instruction-following lost in mid-training without losing the newly acquired knowledge). This estimator (sampled-token log-ratio as a per-token advantage) is exactly the one the MOPD line productionized (MiMo-V2-Flash, then Nemotron; Section 3.3), and is audited in Section 4.

The 2026 on-policy-distillation wave. Within a year the recipe hardened into standard frontier practice (Section 3.3) and acquired its own literature. A survey formalizes OPD as f-divergence minimization over student-generated data (Song, Zheng and others, 2026); entropy-aware variants respond to the finding that reverse KL "reduces generation diversity and yields unstable learning signals when the teacher distribution has high entropy" (quoted in Section 2.3) (Jin *et al.*, 2026); and two failure-mode studies delimit the recipe. Liu *et al.* (2026) identify supervision-fidelity decay:

> ...as student-generated prefixes lengthen, the teacher's next-token distribution becomes less confident and less discriminative. Consequently, the teacher-dependent corrective signal in reverse-KL distillation weakens, causing student drift to compound across long reasoning chains.
>
> — (Liu *et al.*, 2026) · *Your Teacher Can't Help You Here: Combating Supervision Fidelity Decay in On-Policy Distillation*

and Li *et al.* (2026) delimit when OPD works at all:

> We observe a striking failure mode: a stronger teacher can completely fail to improve a student, even when a weaker teacher succeeds from lower initial alignment.
>
> — (Li *et al.*, 2026) · *Rethinking On-Policy Distillation of Large Language Models: Phenomenology, Mechanism, and Recipe*

with the governing conditions:

> ...(i) the student and teacher should share compatible thinking patterns; and (ii) even with consistent thinking patterns and higher scores, the teacher must offer genuinely new capabilities beyond what the student has seen during training.
>
> — (Li *et al.*, 2026) · *Rethinking On-Policy Distillation of Large Language Models: Phenomenology, Mechanism, and Recipe*

3.2 RL and alignment

The regularizer lineage. In the canonical RLHF pipeline the reverse KL to a reference policy enters as a penalty in the **reward channel** of PPO: $R(x, y) = r(x, y) - \beta \log[\pi(y|x)/\rho(y|x)]$ (Ziegler *et al.*, 2019). The direction is never argued against alternatives in this lineage; the justifications are operational:

> This term has several purposes: it plays the role of an entropy bonus, it prevents the policy from moving too far from the range where r is valid, and ... it also is an important part of the task definition.
>
> — (Ziegler *et al.*, 2019) · *Fine-Tuning Language Models from Human Preferences*

> First, it acts as an entropy bonus, encouraging the policy to explore and deterring it from collapsing to a single mode. Second, it ensures the policy doesn't learn to produce outputs that are too different from those that the reward model has seen during training.
>
> — (Stiennon *et al.*, 2020) · *Learning to Summarize from Human Feedback*

Note the historical irony: relative to **unregularized** reward maximization, the reverse-KL penalty was introduced to **prevent** collapse; the later literature blames the same term for collapse relative to **distribution-matching** alternatives (Section 7). InstructGPT makes the penalty per-token:

> In addition, we add a per-token KL penalty from the SFT model at each token to mitigate over-optimization of the reward model.
>
> — (Ouyang *et al.*, 2022) · *Training Language Models to Follow Instructions with Human Feedback*

Anthropic, observing that reward grows linearly in $\sqrt{\mathrm{KL}}$, barely needed the term at all:

> In practice we use a very small value of $\lambda_{\mathrm{KL}} = 0.001$, which likely has a very minor impact during most of RL training (as $D_{\mathrm{KL}} < 100$ typically), and might actually be wholly unnecessary.
>
> — (Bai *et al.*, 2022) · *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*

Gao, Schulman and Hilton (2023) later reported a negative result for the penalty's protective value:

> KL penalty ineffectiveness. In our reinforcement learning setup, using a KL penalty increases the proxy reward model score that can be achieved for a given KL divergence, but this does not correspond to a measurable improvement in the gold RM score–$\mathrm{KL}_{\mathrm{RL}}$ frontier. However, we note this result could be particularly sensitive to hyperparameters.
>
> — (Gao, Schulman and Hilton, 2023) · *Scaling Laws for Reward Model Overoptimization*

The variational view supplies the principled justification of the direction after the fact: the KL-regularized objective is an ELBO, i.e. exactly reverse-KL variational inference toward the tilted posterior, "$J_{\text{KL-RL}}(\theta) \propto -D_{\mathrm{KL}}(\pi_\theta, \pi^*_{\text{KL-RL}})$" with $\pi^* \propto \pi_0 \exp(r/\beta)$ (Korbak, Perez and Buckley, 2022) — reverse because π_θ is the variational approximation. DPO inherits precisely this reverse-KL-constrained objective in closed form, crediting the constraint broadly:

> The added constraint is important, as it prevents the model from deviating too far from the distribution on which the reward model is accurate, as well as maintaining the generation diversity and preventing mode-collapse to single high-reward answers.
>
> — (Rafailov *et al.*, 2023) · *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*

f-DPO observes reverse KL is also **algebraically** special in DPO-style derivations (the partition function cancels) (Wang *et al.*, 2024). Two later results complicate the DPO picture. EXO formalizes the RLHF-as-reverse-KL equivalence and then argues the opposite for DPO itself:

> ...we formally demonstrate that the DPO objective corresponds to the forward KL which is essentially different from the reverse KL required by the alignment objective.
>
> — (Ji *et al.*, 2024) · *Towards Efficient Exact Optimization of Language Model Alignment*

And SimPO drops the reference anchor altogether ("eliminates the need for a reference model"):

> Although SimPO does not impose KL regularization, we find that a combination of practical factors ensures effective learning from preference data while maintaining generalization, leading to an empirically low KL divergence from the reference model.
>
> — (Meng, Xia and Chen, 2024) · *SimPO: Simple Preference Optimization with a Reference-Free Reward*

GRPO moved the KL from the reward into the loss, using Schulman's k3 estimator per token:

> Also note that, instead of adding KL penalty in the reward, GRPO regularizes by directly adding the KL divergence between the trained policy and the reference policy to the loss, avoiding complicating the calculation of $\hat{A}_{i,t}$.
>
> — (Shao *et al.*, 2024) · *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*

What gradient that actually yields is the subject of Section 4.2. Since 2025, RLVR practice has split three ways over the term:

- **Remove it.** DAPO, under a section titled "Removing KL Divergence":

  > However, during training the long-CoT reasoning model, the model distribution can diverge significantly from the initial model, thus this restriction is not necessary. Therefore, we will exclude the KL term from our proposed algorithm.
  >
  > — (Yu *et al.*, 2025) · *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*

  Dr. GRPO drops it for the same regime reason, plus cost:

  > However, RL-tuning reasoning models typically employs rule-based verifiers as r (Lambert et al., 2024), eliminating the concerns of distributional shift. This allows us to remove the KL term, which not only saves the memory and computation required by π_{ref} during training, but also potentially leads to better performance for R1-Zero-like training (Hu et al., 2025).
  >
  > — (Z. Liu *et al.*, 2025) · *Understanding R1-Zero-Like Training: A Critical Perspective*

  Magistral reaches the same verdict:

  > Eliminating KL divergence. The KL divergence penalty constrains the online policy from deviating too much from a reference policy, helping to maintain alignment with the initial model. However, in GRPO, the policy diverges substantially regardless, and maintaining a copy of the reference model for KL computation incurs a compute cost we find unjustified. We remove the KL penalty entirely.
  >
  > — (Mistral AI, 2025) · *Magistral*

  Nemotron-Cascade 2 does the same ("we remove KL divergence term entirely" (NVIDIA, 2026a)), and GSPO brackets it out without argument ("we omit the KL regularization term hereinafter for brevity, as it is not the focus of this paper" (Zheng *et al.*, 2025)).
- **Keep and repair it.** DeepSeek-V3.2 retains the penalty, corrects the k3 estimator with an importance-sampling ratio (Section 4.2), and notes that "different domains benefit from varying strengths of KL regularization" (DeepSeek-AI, 2025); FRPO repairs its per-token credit assignment with a future-KL return-to-go (Yao *et al.*, 2026) (Section 4.2).
- **Re-introduce it.** ProRL brings it back (with reference resets) precisely to **fight** entropy collapse (Liu *et al.*, 2025a).

3.3 Production technical reports

The sweep of technical reports (full dossier: `notes/D`) yields a clean pattern, summarized below. The type column combines the direction (our convention; "unspecified" means the report's text does not determine it) with the estimator support (full-vocab / truncated-support / sampled-token); the next column records how the divergence enters the training loss.

Report / stage	Objective (as stated)	Type	Introduced via	Sampled from
MiMo-V2-Flash — MOPD (Xiaomi LLM-Core Team, 2026)	introduces MOPD: "reverse KL divergence loss" (Eq. 5)	sampled-token **reverse KL** (explicit)	gated $\text{sg}[\log(\pi_{\text{domain}}/\pi_\theta)]$ as per-token advantage in a length-normalized	student rollouts

Report / stage	Objective (as stated)	Type	Introduced via	Sampled from
			REINFORCE-style update (Eqs. 7–9)	
Nemotron 3 Ultra — MOPD (NVIDIA, 2026b)	explicit "negative reverse-KL objective" (Eq. 1)	sampled-token **reverse KL** (explicit)	$\mathrm{sg}\left[\log\left(\pi_{T_i}/\pi_{\text{prox}}\right)\right]$ as per-token advantage in a PPO-clipped update with IcePop masking (Eqs. 2–3)	student rollouts
Nemotron-Cascade 2 — MOPD (NVIDIA, 2026a)	"token-level distillation advantage using reverse-KL … only on the student-sampled token"	sampled-token **reverse KL** (explicit)	per-token advantage in a truncated-IS, length-normalized REINFORCE update (Eqs. 2–4)	student rollouts
Nemotron 3 Ultra — MTP head	"temperature-scaled forward-KL loss" (T=2)	full-vocab forward KL	directly optimized (auxiliary MTP loss; CE to the gold token disabled)	own rollouts
Qwen3 — off-policy distillation (Qwen Team, 2025)	"outputs of teacher models … for response distillation"	SFT on teacher samples (no KL)	—	teacher
Qwen3 — on-policy distillation (Qwen Team, 2025)	"aligning its logits with those of a teacher model … to minimize the KL divergence"	logit-level KL, direction unspecified	directly optimized (no equation given)	student rollouts
DeepSeek-V4 — consolidation OPD (DeepSeek-AI, 2026)	"Computing the reverse KL loss … requires sampling training trajectories from the student" (Eq. 29; multi-teacher)	**full-vocab reverse KL** (explicit)	directly optimized (exact per-position KL via TileLang kernel; the per-token-advantage route explicitly rejected)	student rollouts
Kimi K3 — MOPD (Kimi Team, 2026)	MOPD "to consolidate these domain-specialized capabilities … into a unified model"	sampled-token **reverse KL** (implicit)	per-token reward $\mathrm{sg}[\log(\pi_{\text{teacher}}/\pi_\theta)]$ clipped to $\pm R_{\max}$ (Eq. 15)	student rollouts
GLM-5 — cross-stage distillation (GLM Team, 2026)	on-policy cross-stage distillation; earlier-stage checkpoints as teachers	sampled-token **reverse KL** (implicit)	per-token advantage $\hat{A}_{i,t} = \mathrm{sg}[\log(\pi_{\text{teacher}}/\pi_\theta)]$	student rollouts
DeepSeek-V3.2 — RL (GRPO) (DeepSeek-AI, 2025)	KL penalty kept; k3 estimator corrected by importance ratio (Eq. 7)	sampled-token **reverse KL** (regularizer)	importance-ratio-corrected k3 in the GRPO loss (cf. Eq. 16)	student
Magistral — RLVR (Mistral AI, 2025)	"We remove the KL penalty entirely"	removed	—	—
Llama 4 — pretraining codistillation (Meta AI, 2025)	Maverick from Behemoth; "novel distillation loss function that dynamically weights the soft and hard targets"	unspecified	directly optimized	dataset
Gemma 2 — pretraining KD (Gemma Team, 2024)	$\min_{P_S} \sum_x -P_T(x\|x_c) \log P_S(x\|x_c)$	full-vocab forward KL (CE to soft targets)	directly optimized (pretraining loss)	dataset
Gemma 2 — post-training (Gemma Team, 2024)	"distillation from the teacher on the student's distribution", citing GKD and MiniLLM	unspecified	unspecified	student
Gemma 3 — pretraining KD (Gemma Team, 2025)	CE on 256 sampled logits, renormalized	truncated-support forward KL (256 sampled logits)	directly optimized (pretraining loss)	dataset
Minitron (Muralidharan *et al.*, 2024)	logit KD; "KLD is the best choice"; R-KLD ablated and worse	full-vocab forward KL (reverse tried, failed)	directly optimized ($\mathcal{L}_{\text{logits}}$ term)	dataset
Nemotron-H / Nano 2 (NVIDIA, 2025a; 2025b)	"forward KL divergence loss exclusively"	full-vocab forward KL	directly optimized (Minitron loss formulation)	dataset

Report / stage	Objective (as stated)	Type	Introduced via	Sampled from
Llama 3.2 1B/3B (Meta AI, 2024)	teacher "logits … used as token-level targets"	logit-level, direction unspecified	unspecified (blog-level)	dataset
DeepSeek-R1 distillation (DeepSeek-AI, Guo and others, 2025)	SFT on 800k teacher-generated samples, "only SFT … no RL stage"	SFT on teacher samples (no KL)	—	teacher
Apple AFM 2024 — pretraining (Apple, 2024)	teacher top-1 labels mixed 0.9 with true labels	top-1 (hard-label) forward-flavored	directly optimized (CE to label mix)	dataset
Apple AFM 2024 — RLHF (MDLOO) (Apple, 2024)	$r_\varphi(x,y) - \beta D_{\mathrm{KL}}(\pi_\theta(\cdot\|x) \,\|\, \pi_{\mathrm{ref}}(\cdot\|x))$ — direction written out	**reverse KL** (regularizer; estimator unstated)	β-penalty on the reward in the RL objective	student
Gemini 1.5 Flash (Gemini Team, 2024)	"online distilled" from 1.5 Pro (cites GKD)	unspecified	unspecified	unspecified
Tülu 3 / OLMo 2 (Lambert *et al.*, 2024; OLMo Team, 2025)	$r_\varphi - \beta\ \mathrm{KL}[\pi_\theta \,\|\, \pi_{\mathrm{ref}}]$ (boilerplate)	**reverse KL** (regularizer; estimator unnamed)	penalty in reward as written; k1 in PPO reward in paper-era open-instruct code	student

Reports checked with **no** KD/KL distillation stage described: Llama 3, Phi-4, MiniCPM, Hunyuan-Large/TurboS, GLM-4, SmolLM2, InternLM2, Yi, OLMo 2 (which explicitly notes distillation as a road not taken), Kimi K2 (whose OPMD objective is a squared log-ratio to the previous policy, not a KL) (Kimi Team, 2025), ERNIE 4.5, Falcon 3 (blog-level only), Ministral, Grok, GPT-4o mini.

Three regularities stand out. First, **explicit reverse-KL training appears only in on-policy post-training distillation** — the MOPD paradigm is introduced by Xiaomi's MiMo-V2-Flash (January 2026), with an explicit "reverse KL divergence loss" estimated by a gated sampled-token advantage in a length-normalized REINFORCE-style update (its Eqs. 5–9) (Xiaomi LLM-Core Team, 2026); the Nemotron MOPD stages ship the same sampled-token estimator, "computed only on the student-sampled token rather than over the full vocabulary" (NVIDIA, 2026a), and NVIDIA's public implementation cites MiMo's Eq. 8 for its advantage (Section 4.5); by mid-2026 the recipe is standard frontier practice: DeepSeek-V4 replaces its mixed-RL consolidation stage with explicit multi-teacher reverse-KL distillation over full-vocabulary logits (DeepSeek-AI, 2026), while Kimi K3's MOPD and GLM-5's cross-stage distillation ship the same objective implicitly, as a stop-gradient log-ratio advantage on student rollouts (Kimi Team, 2026; GLM Team, 2026). Second, **pretraining-scale distillation is uniformly forward-KL or unspecified** (Gemma 2/3, Minitron line, Apple, Llama 3.2). Third, the reverse-KL RLHF regularizer is universal but boilerplate; almost no report specifies the estimator — DeepSeek-V3.2, which specifies **and corrects** its k3 estimator, is the exception (DeepSeek-AI, 2025).

Failure reports. The most informative quote in the sweep is Nemotron 3 Ultra's §3.3.5 — a negative result for **full-distribution** reverse-KL distillation on student prefixes, from the team whose shipped objective is reverse KL at sampled tokens:

> A natural alternative to the sampled-token objective is distribution-level distillation, where the student is trained to match the teacher's predictive distribution over the top-k tokens or the full vocabulary at each prefix. In our preliminary experiments, these objectives did not improve MOPD performance and consistently underperformed the sampled-token objective on some agentic benchmarks such as Terminal Bench. We hypothesize that full-distribution matching may impose an overly strong local constraint on prefixes sampled from the student policy, particularly when those prefixes have limited support under the teacher distribution. In this regime, teacher logits can become poorly calibrated or less informative, and matching the full distribution may amplify noise from off-support states.
>
> — (NVIDIA, 2026b) · *Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning*

Note what failed and what did not: the divergence (reverse KL) is retained; the **estimator** (full-vocabulary or top-k integrand versus sampled token) is what made the difference — the report does not say whose top-k; NVIDIA's public top-k path, nemo-rl's `DistillationLossFn`, takes the ids from the **teacher** (Section 4.6). The question is not settled at the frontier: DeepSeek-V4 ships exactly the configuration Nemotron found inferior — full-vocabulary

reverse KL on student rollouts — as its consolidation stage (DeepSeek-AI, 2026). The complementary production negative result is Minitron's, in the opposite regime (teacher-forced base-model compression), with a large gap (LM loss 2.155 forward vs 2.665 reverse for the depth-pruned 8B):

> ...we experiment with Kullback-Leibler divergence (KLD), MSE, cosine similarity and reverse KLD (R-KLD) ... We observe from Table 15 and 16 that KLD is the best choice for pruned base model training.
>
> — (Muralidharan *et al.*, 2024) · *Compact Language Models via Pruning and Knowledge Distillation*

Apple reports a direction-agnostic negative:

> We did not find a distillation loss to be helpful here [continued pretraining] for AFM-on-device, unlike in core pre-training.
>
> — (Apple, 2024) · *Apple Intelligence Foundation Language Models*

3.4 Student-sampled objectives

Student-sampled stages from the production reports (Section 3.3) and the estimator audit (Section 4.3), grouped by estimator family. Parentheses mark the support the KL is taken over.

rkl_pg	rkl	fkl
MiniLLM - KD (sampled-token) Tinker - OPD (sampled-token) MiMo-V2-Flash - MOPD Nemotron 3 Ultra - MOPD (attempted teacher-top-k) Nemotron-Cascade 2 - MOPD Kimi K3 - MOPD GLM-5 - cross-stage distillation InstructGPT / Ziegler - RLHF (sampled-token) Tülu 3 / OLMo 2 (within RL) DeepSeekMath - GRPO (sampled-token) DeepSeek-V3.2 - (within GRPO)	GKD - KD (full-vocab) DistiLLM - SRKL (full-vocab) DistiLLM-2 - SRKL (full-vocab) DeepSeek-V4 - OPD (full-vocab)	

4 Estimators and implementations

This section is the core of the review: for each implementation associated with a reverse-KL training stage, what expression is computed, what is sampled, which gradient is taken, and whether the result is consistent with the claimed objective. Code references are commit-pinned; classifications use the categories **correct for the claimed objective** / **correct for a stated surrogate** / **biased or inconsistent with the claimed objective** / **indeterminate**, each with a confidence level.

4.1 Taxonomy: which "reverse KL" is being estimated

Write $s_t = (x, y_{<t})$ for a prefix and $q_\theta(\cdot\,|s_t)$, $p(\cdot\,|s_t)$ for next-token distributions. The chain rule gives the exact identity

$$D_{\mathrm{KL}}(q_\theta(y|x) \,\|\, p(y|x)) = \sum_t \mathbb{E}_{s_t \sim q_\theta}[D_{\mathrm{KL}}(q_\theta(\cdot\,|s_t) \,\|\, p(\cdot\,|s_t))], \tag{7}$$

so **sequence-level** reverse KL equals the sum of **token-level conditional** KLs weighted by the student's own prefix-visitation distribution. This equivalence is exact for the objective **value** — but only when prefixes are drawn from q_θ. When prefixes come from a fixed dataset (GKD's off-policy fraction, DistiLLM, torchtune, all pretraining KD) or from the teacher, the summed token-level KL is a **different** objective: an expected conditional divergence under a mismatched visitation distribution, which neither equals nor generally bounds the sequence-level KL. Independently, each token-level KL may be computed **exactly over the full vocabulary** or estimated **at sampled tokens**; and the whole construct may serve as **primary objective** (distillation), **regularizer added to the loss** (GRPO-style), or **penalty added to the reward** (PPO-style). A reward-channel penalty and a loss-channel term describe the same objective value, but their gradients differ by construction — the subject of the next subsection. Finally, note that minimizing sequence-level reverse KL is **identical** to entropy-regularized reward maximization

with reward $\log p$ (up to constants, $\min_\theta \mathrm{KL}(q \,\|\, p) = \max_\theta \mathbb{E}_q[\log p] + \mathcal{H}(q)$) — which is why RL machinery attaches so naturally to reverse-KL distillation (MiniLLM, Tinker, MOPD).

4.2 Derivations: values versus gradients

All derivations are for on-policy sampling, $y \sim q_\theta$, with p fixed; write $r_t = p(y_t|s_t)/q_\theta(y_t|s_t)$ for the teacher/student probability ratio at the sampled token, so the per-token reverse KL is $D_{\mathrm{KL}}(q_\theta(\cdot\,|s_t) \,\|\, p(\cdot\,|s_t)) = \mathbb{E}_{y_t\sim q_\theta}[-\log r_t]$. Schulman proposes three single-sample estimators of this **value**, each computable from the sampled token alone (Schulman, 2020):

$$k1_t = -\log r_t = \log q_\theta(y_t|s_t) - \log p(y_t|s_t), \tag{8}$$

$$k2_t = (\log r_t)^2/2 = (k1_t)^2/2, \tag{9}$$

$$k3_t = r_t - 1 - \log r_t = (r_t - 1) + k1_t. \tag{10}$$

$k1$ is the naive Monte Carlo estimate — the log-ratio itself: unbiased by construction, but it can be negative even though the KL never is, and its variance is large when q_θ and p are far apart. $k2$ replaces the log-ratio by half its square: always nonnegative and low-variance, but biased for the reverse KL — the squared log-ratio matches the KL only to second order in $\log r$, so the bias vanishes as $q_\theta \to p$. $k3$ adds to $k1$ the control variate $r_t - 1$, which has expectation zero under q_θ (since $\mathbb{E}_q[r] = 1$): it remains unbiased, and is pointwise nonnegative because $\log r \le r - 1$, which is what makes it low-variance. The blog is explicit that these are diagnostics for the value. Whether their **derivatives** estimate the reverse-KL gradient is a separate question (Tang and Munos, 2025; Liu *et al.*, 2025b; Zhang *et al.*, 2025).

The master identity is, for any per-sample integrand f_θ,

$$\nabla_\theta \mathbb{E}_{y\sim q_\theta}[f_\theta(y)] = \mathbb{E}[\nabla_\theta f_\theta(y)] + \mathbb{E}[f_\theta(y)\nabla_\theta \log q_\theta(y)], \tag{11}$$

a pathwise term plus a score-function term. With $f_\theta = \log q_\theta - \log p$ the pathwise term vanishes in expectation ($\mathbb{E}[\nabla \log q] = 0$), leaving the score-function-only form $\nabla\ \mathrm{KL} = \mathbb{E}[(\log q - \log p)\nabla \log q]$; per token, using $\mathbb{E}[\nabla \log q] = 0$ again,

$$\nabla_\theta D_{\mathrm{KL}}(q_\theta \,\|\, p) = \mathbb{E}_{y_t\sim q_\theta(\cdot|s_t)}[k1_t \cdot \nabla_\theta \log q_\theta(y_t \mid s_t)]. \tag{12}$$

Naive backprop is zero-mean noise. The estimator "sample $y \sim q$, compute $\log q_\theta(y) - \log p(y)$, backpropagate" gives an unbiased estimate of the **value** of the KL, but its gradient is $\nabla \log q_\theta(y)$, whose expectation is 0: the estimator's expected gradient contains none of the score-function weighting in Eq. 12. (a) correct for the value; (b) pure noise for the gradient. This is Tang and Munos's first pitfall:

> ...such implementations are generally incorrect and do not produce the desired KL gradient.
>
> — (Tang and Munos, 2025) · *On a Few Pitfalls in KL Divergence Gradient Estimation for RL*

The three k-estimators as losses. Differentiating each estimator through $\log q_\theta$ at the sampled token:

$$\nabla k1 = \nabla \log q, \quad \mathbb{E}[\nabla k1] = 0; \tag{13}$$

$$\nabla k2 = (\log q - \log p)\nabla \log q = k1 \nabla \log q, \quad \mathbb{E}[\nabla k2] = \nabla D_{\mathrm{KL}}(q \,\|\, p); \tag{14}$$

$$\nabla k3 = (1 - r)\nabla \log q, \quad \mathbb{E}[\nabla k3] = -\sum_v p(v)\nabla \log q(v) = \nabla D_{\mathrm{KL}}(p \,\|\, q). \tag{15}$$

The $k3$ line uses $\mathbb{E}_q[r\nabla \log q] = \sum_v p(v)\nabla \log q(v) = -\nabla H(p, q)$. So ***k*2-as-loss is the on-policy-correct differentiable surrogate for reverse KL, while *k*3-as-loss — the GRPO pattern — has an expected gradient pointing along the forward KL**:

> ...the positions of the two distributions are reversed compared to the target KL divergence ... the regularization effect of the variance reduced estimate is completely incidental.
>
> — (Tang and Munos, 2025) · *On a Few Pitfalls in KL Divergence Gradient Estimation for RL*

It is still a penalty pulling q toward p (which is why it "works" as a regularizer), but a mass-covering one, not the mode-seeking one on the label.

Repairing k3 with a differentiable ratio. Multiplying $k3$ by the importance ratio $\rho = q_\theta / \operatorname{sg}[q_\theta]$ (value 1, gradient $\nabla \log q$) — the correction introduced by DeepSeek-V3.2 (their Eq. 7, an importance-sampling fix with π_{old} in the denominator) and implemented by TRL's `use_bias_correction_kl` and NeMo-RL's surrogate — restores the reverse direction: $\nabla(\rho k3) = k3 \nabla \log q + \nabla k3$, and since $\mathbb{E}[k3 \nabla \log q] = \mathbb{E}[(r-1)\nabla \log q] + \mathbb{E}[k1 \nabla \log q] = -\nabla D_{\mathrm{KL}}(p \,\|\, q) + \nabla D_{\mathrm{KL}}(q \,\|\, p)$, the totals cancel to

$$\mathbb{E}[\nabla(\rho k3)] = \nabla D_{\mathrm{KL}}(q \,\|\, p). \tag{16}$$

DeepSeek-V3.2 motivates the fix by the uncorrected estimator's behavior when $\pi_\theta \ll \pi_{\text{ref}}$:

> In such cases, the gradient of the K3 estimator assigns disproportionately large, unbounded weights to maximize the likelihood of these tokens, resulting in noisy gradient updates that accumulate to degrade sample quality in subsequent iterations and lead to unstable training dynamics.
>
> — (DeepSeek-AI, 2025) · *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*

KL in the reward channel. Subtracting $\beta k1_t$ from the per-token reward and applying an idealized (critic-free, reward-to-go, unclipped) policy gradient contributes $-\beta \mathbb{E}\left[\sum_t \nabla \log q(y_t|s_t) \sum_{t' \geq t} k1_{t'}\right]$, which is exactly the score-function form of $-\beta \nabla D_{\mathrm{KL}}$ for the **sequence-level** KL, including the effect of θ on prefix visitation. Liu *et al.* (2025b) state that on-policy, "the 'k2 as loss' formulation is, in fact, gradient-equivalent to 'k1 in reward'"; by the derivations here the equivalence is exact for each token's **own** penalty term ($k1_t \nabla \log q_t$, cf. Eq. 14), while the reward-to-go channel **additionally** propagates each $k1_{t'}$ into the score functions of earlier tokens $t < t'$ — precisely the visitation cross-terms that separate the sequence-level gradient from the frozen-visitation one, and which loss-channel $k2$ does not carry. Off-policy, every variant needs importance-weight corrections, which RPG derives systematically, including "an off-policy importance-weighting mismatch in GRPO's KL term" (Zhang *et al.*, 2025). In deployed PPO the idealization is broken by GAE, critics, whitening, and clipping — the objective family is right, the exact gradient is not.

Token-level versus sequence-level: the visitation term. For fixed x, let $d_{\theta,t}(s_t) = q_\theta(y_{<t}|x)$ be the distribution of the prefix s_t, and abbreviate the conditional divergence by

$$D_t(s_t;\theta) = D_{\mathrm{KL}}(q_\theta(\cdot\,|s_t) \,\|\, p(\cdot\,|s_t)). \tag{17}$$

Then Eq. 7 is the objective

$$\mathcal{L}_{\text{seq}(\theta)} = \sum_t \mathbb{E}_{s_t \sim d_{\theta,t}}[D_t(s_t;\theta)]. \tag{18}$$

Applying Eq. 11 to the outer expectation gives the exact split

$$\nabla_\theta \mathcal{L}_{\text{seq}} = \underbrace{\sum_t \mathbb{E}_{s_t \sim d_{\theta,t}}[\nabla_\theta D_t(s_t;\theta)]}_{G_{\text{local}}} + \underbrace{\sum_t \mathbb{E}_{s_t \sim d_{\theta,t}}\left[D_t(s_t;\theta) \nabla_\theta \log d_{\theta,t}(s_t)\right]}_{G_{\text{visit}}}. \tag{19}$$

The first term is the gradient obtained with prefix visitation frozen. More formally, if $|\theta|$ denotes the fixed parameters used to sample prefixes, then

$$\mathcal{L}_{\text{frz}}(\theta;|\theta|) = \sum_t \mathbb{E}_{s_t \sim d_{|\theta|,t}}[D_t(s_t;\theta)], \quad G_{\text{local}} = \nabla_\theta \mathcal{L}_{\text{frz}}(\theta;|\theta|)|_{|\theta|=\theta}. \tag{20}$$

The prefix score expands over earlier tokens:

$$\nabla_\theta \log d_{\theta,t}(s_t) = \sum_{u<t} \nabla_\theta \log q_\theta(y_u|s_u). \tag{21}$$

Since $\mathbb{E}[k1_t \mid s_t] = D_t(s_t;\theta)$ and Eq. 12 holds conditional on s_t, both terms have sampled-token forms:

$$G_{\text{local}} = \mathbb{E}_{y \sim q_\theta}\left[\sum_t k1_t \nabla_\theta \log q_\theta(y_t|s_t)\right], \tag{22}$$

$$G_{\text{visit}} = \mathbb{E}_{y\sim q_\theta}\left[\sum_t k1_t \sum_{u<t} \nabla_\theta \log q_\theta(y_u|s_u)\right]. \tag{23}$$

Combining them and exchanging the two sums yields the full sequence gradient:

$$\nabla_\theta \mathcal{L}_{\text{seq}} = \mathbb{E}_{y\sim q_\theta}\left[\sum_u \nabla_\theta \log q_\theta(y_u|s_u) \sum_{t\geq u} k1_t\right]. \tag{24}$$

Thus implementations that sample prefixes from the student but treat them as data (GKD's full-vocabulary loss; Tinker's and MOPD's per-token advantages without reward-to-go) compute G_{local} only — "a partial gradient at best" in Tang and Munos's phrase, though for GKD this is the **stated** objective, not an error. Indeed, under the usual policy-gradient loss convention, the local advantage

$$A_t^{\text{KL}} = -\text{sg}[k1_t], \quad \nabla_{\theta[-A_t^{\text{KL}} \log q_\theta(y_t|s_t)]} = k1_t \nabla_\theta \log q_\theta(y_t|s_t) \tag{25}$$

is unbiased for G_{local}. Replacing it by the undiscounted return-to-go,

$$A_u^{\text{KL}} = -\sum_{t\geq u} \text{sg}[k1_t], \tag{26}$$

restores $G_{\text{local}} + G_{\text{visit}} = \nabla_\theta \mathcal{L}_{\text{seq}}$. The shipped defaults (Tinker `kl_discount_factor = 0.0`) deliberately omit this visitation term. The omission is Tang and Munos's second pitfall, and their §4.1 spells out the sense in which the result is "partial":

> Secondly, we show that some implementations do not account for the sequential nature of the estimation problem and produce a partial gradient at best. ... Importantly, the above gradient [the summed per-token gradient estimates] does not account for the impact that token y_t has on the future tokens y_s, $s > t$ which impacts future token-level KL divergences. Henceforth, we call the above loss implementation the token-level losses, which constitute estimates to partial gradient of sequence-level KL divergence...
>
> — (Tang and Munos, 2025) · *On a Few Pitfalls in KL Divergence Gradient Estimation for RL*

Their Appendix D derives the same split as above, its "part i" and "part ii" being exactly G_{local} and G_{visit}.

The dual partial gradient: exact KL in the reward channel. The complementary omission arises when the **exact** per-position KL $D_t(s_t;\theta)$ — not its sampled-token estimate $k1_t$ — is placed in the reward under stop-gradient. Because D_t is a deterministic function of the prefix, it carries no dependence on the token sampled at t: the own-position score term vanishes, $\mathbb{E}[D_t(s_t)\nabla_\theta \log q_\theta(y_t|s_t)] = \mathbb{E}[D_t(s_t)\cdot \mathbb{E}[\nabla_\theta \log q_\theta(y_t|s_t) \mid s_t]] = 0$, and the stop-gradient blocks the pathwise route through $q_\theta(\cdot\,|s_t)$. What survives of the idealized reward-to-go policy gradient is $-\beta G_{\text{visit}}$ alone: derandomizing the penalty destroys exactly the sampled-token dependence that lets the score function recover G_{local} from $k1$-in-reward — the same mechanism by which naive backprop of $k1$ loses everything. Token-level losses keep G_{local} and drop G_{visit}; exact-KL-in-reward keeps G_{visit} and drops G_{local}. Tang and Munos do not treat this case; it matters here because trl-classic ships it as the opt-in `"full"` penalty (Section 4.3, Section 4.5).

For GKD, by contrast, computing only G_{local} is the announced design — the on-policy loss (their Eq. 4) is defined with the sampling distribution outside the gradient:

> ...where we do not backpropagate through the student's sampling distribution $p_S(\cdot\,|x)$, similar to on-policy imitation. Not backpropagating through the sampling makes the training stable and computationally efficient.
>
> — (Agarwal *et al.*, 2024) · *On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes*

and the paper defends the choice as an advantage over MiniLLM's policy-gradient treatment, which does carry the score-function machinery:

> …we argue that GKD is simpler and more stable, being closer to supervised training, since it does not backpropagate through the student's sampling process. Indeed, MiniLLM relies on a number of stabilizing tricks, to tackle high variance, reward hacking, and generation length bias.
>
> — (Agarwal *et al.*, 2024) · *On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes*

FRPO subsequently made exactly this correction its contribution:

> The reward term recovers the standardized GRPO advantage, while the regularizer term includes a causal future-regularization return-to-go omitted by local KL losses. For reverse KL, this yields a simple future KL correction: add a reverse cumulative sum of per-token log ratios after advantage construction.
>
> — (Yao *et al.*, 2026) · *Future-KL Regularized GRPO: Process-Level Credit Assignment from f-Divergence Regularization*

4.3 Estimator audit table

Table 1 — Estimator audit. q student/policy (trained), p teacher/reference; $k1$, $k2$, $k3$ as in Section 4.2; sg = stop-gradient; seq = sequence-level, tok = token-level conditioned on prefixes; full-vocab = summed over the full vocabulary, sampled-token = estimated at the sampled token only. Classifications: **C** correct for the claimed objective; **S** correct for a stated surrogate; **B** biased/inconsistent with the claimed objective; **I** indeterminate. Justifications for non-trivial rows follow in Section 4.5.

Occurrence	Stage	Claimed objective & stated surrogate	Expression computed	Sampled from	Support mods	Resulting objective / $\mathbb{E}[\nabla]$	Class.
MiniLLM (Gu *et al.*, 2024)	KD fine-tune	• seq-level RKL • surrogate stated: PPO-style PG with three named stabilizing techniques[1]	$\mathcal{L} = \mathcal{L}_{\text{PG}} + \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{PT}}$ $\mathcal{L}_{\text{PG}} = -\frac{1}{N}\sum_t \min(\rho_t A_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)A_t)$ • **importance ratio** $\rho_t = \frac{q_\theta(y_t\|s_t)}{q_{\text{old}}(y_t\|s_t)} \cdot \text{sg}\left[\frac{q_{\text{old}}(y_t\|s_t)}{p_{\text{sample}}(y_t\|s_t)}\right]$[2] • **advantage** $A_t = \text{sg}\left[\text{whiten}\left(\left(\sum_{t'\geq t}\gamma^{t'-t} r_{t'}\right)/L_t\right)\right]$ whitened return-to-go; no critic • **per-token reward** $r_t = \log p(y_t\|s_t) - \log q_{\text{old}}(y_t\|s_t)$ $\mathcal{L}_{\text{reg}} = \frac{1}{N}\sum_t D_{\text{KL}}(q_\theta(\cdot\|s_t) \parallel p(\cdot\|s_t))$ full-vocab, same rollout prefixes $\mathcal{L}_{\text{PT}} = \mathbb{E}_{\text{OpenWebText}}\left[\frac{1}{2}(-\log q_\theta(y_t\|s_t)) + \frac{1}{2}\left(-\sum_v p(v\|s_t)\log q_\theta(v\|s_t)\right)\right]$ 2nd term = FKL + const. • ∇ through live q_θ in ρ_t, $\mathcal{L}_{\text{reg}}$, $\mathcal{L}_{\text{PT}}$ only Where: • N = # response tokens in batch • L_t = remaining response length	$y \sim p_{\text{sample}}(\cdot\|s_t)$ $= (1-\alpha)q_{\text{old}}(\cdot\|s_t)$ $+\alpha p(\cdot\|s_t)$ $\alpha = 0.2$	• reward scaling 0.5 • reward clip ± 100 • PPO clip 0.2 • $\gamma = 0.95$ • whitening • length norm • teacher-mix sampling	• seq-PG + tok-level full-vocab: variance-reduced, reward-shaped PG surrogate of $\nabla \text{KL}_{\text{seq}}(q \parallel p)$ − visitation term carried by the reward-to-go ($\gamma = 0.95$ discounts it) • local term double-counted: A_t keeps the own-token reward **and** the exact single-step term is added (the paper's Alg. 1 assigns R_{t+1} to the PG part; Section 4.5) • deviations deliberate but bias the gradient • per-token-only IS correction for mixture prefixes is an extra approximation	• **B** (mild; med) • S for the stated PPO composite; B because the shipped estimator deviates unstated from the paper's own Alg. 1 (R_{t+1} stated; R_t + exact term shipped)
GKD (Agarwal *et al.*, 2024) **Frameworks:** trl `GKDTrainer` ($\beta = 1$)	KD fine-tune	• tok-level generalized JSD: $\text{JSD}(\beta) = \beta D_{\text{KL}}(p \parallel m) + (1-\beta)D_{\text{KL}}(q \parallel m)$, $m = \beta p + (1-\beta)q$ • at $\beta = 1$ (audited here) the trainer computes exact RKL $D_{\text{KL}}(q \parallel p)$ • no surrogate — exact computation intended	$\mathcal{L} = \frac{1}{N}\sum_t D_{\text{KL}}(q(\cdot\|s_t) \parallel p(\cdot\|s_t))$ full-vocab, exact • ∇ through live q over the whole vocab only; teacher sg; none through prefix sampling Where: N = # response tokens in batch	prefixes: dataset w.p. $1-\lambda$, student rollouts w.p. λ (per-batch coin)	• generation at $T = 0.9$ but loss at $T = 1$ • top-k/p off	exact token-KL under frozen data distribution — GKD's stated objective	**C** (high)
DistiLLM (Ko *et al.*, 2024)	KD fine-tune (only for SRKL)	• tok-level skew reverse KL (SRKL), adaptive off-policy • no surrogate	• full-vocab $D_{\text{KL}}(q \parallel (1-\alpha)p + \alpha q)$, $\alpha = 0.1$, per position • ∇ through q everywhere incl. inside the mixture; teacher sg	prefixes: dataset / student replay (adaptive)	none	exact skew-RKL (the claimed objective)	**C** (high)
DistiLLM-2 (Ko *et al.*, 2025)	KD fine-tune	• tok-level SKL on teacher data + SRKL on student data • the mixture detach is unstated in the paper	• SKL $D_{\text{KL}}(p \parallel m_1)$, $m_1 = \alpha p + (1-\alpha)q$, on teacher responses ($q$ inside m_1 live) • SRKL $D_{\text{KL}}(q \parallel m_2)$, $m_2 = \alpha q + (1-\alpha)p$, on student responses ($q$ inside m_2 detached) • outer q, $\log q$ live; tok-level full-vocab, per-seq mean	responses generated offline (teacher/student, $T = 0.8$)	fixed $\alpha = 0.1$	• value = claimed • gradient omits a bounded $O(\alpha)$ mixture term that vanishes at the optimum (derivation in Section 4.5)	**B** (mild; med)
Qwen3 on-policy distillation (Qwen Team, 2025)	post-training	• "minimize the KL divergence" by "aligning its logits with those of a teacher model" (Qwen3-32B or Qwen3-235B-A22B) on student-generated sequences (their §4.5, phase 2) — **direction unspecified** • no surrogate — no equation anywhere in the report	• none recoverable: the quoted sentence is the report's entire specification; no training code released • logit-level alignment suggests a full-vocab integrand; direction, token aggregation, and ∇ path all unstated	$y \sim q$ (`/think` or `/no think` mode)	none stated	• indeterminate: full-vocab RKL on student rollouts (the GKD-on-rollouts family) and FKL on the same prefixes are both consistent with the text • cited as an RKL-OPD precedent by later frontier reports (Section 3.3) despite never stating a direction	**I** (high — one sentence, no equation, no code)
Tinker on-policy distillation	post-training	• per-token RKL on student rollouts as a dense reward	$\mathcal{L} = -\sum_t \rho_t A_t$ sampled-token, in-loss via advantage	$y \sim q$, $T = 1$	• no clipping	• unbiased for $\sum_t \mathbb{E}_{s_t}[\nabla D_{\text{KL},t}]$, the frozen-visitation gradient (Section 4.2)	**S** (high)

[1] PPO-style: clipped importance-sampling policy gradient, rollouts from q_{old} reused over several epochs. The three techniques — single-step decomposition, teacher-mixed sampling, length normalization — are detailed in Section 4.5.

[2] q_θ: the live student (current parameters; the only gradient path); q_{old}: the frozen student snapshot that generated the rollouts — the PPO "old" policy, with each batch reused for four inner epochs, so q_θ drifts from it; p_{sample}: the teacher-mixed behavior policy the rollouts are actually drawn from (generation only, never carries gradient).

Occurrence	Stage	Claimed objective & stated surrogate	Expression computed	Sampled from	Support mods	Resulting objective / $\mathbb{E}[\nabla]$	Class.
(Lu and Thinking Machines Lab, 2025)		• IS loss form per official docs	• **importance ratio** $\rho_t = \frac{q_\theta(y_t\|s_t)}{q_{\text{sample}}(y_t\|s_t)}$[3] • **advantage** $A_t = -\text{sg}\left[\log q_{\text{sample}}(y_t\|s_t) - \log p(y_t\|s_t)\right]$ • ∇ through live q_θ in ρ_t only		• teacher scores the sampled sequence	• omits visitation term (discount 0)	
Nemotron 3 Ultra MOPD (NVIDIA, 2026b)	MOPD	• seq-level RKL (but token-level surrogate) — per-token RKLs summed along the student rollout — to domain teachers p_{T_i}, mixed with weights λ_i (their Eq. 1) • surrogate stated: async PPO-clipped update with IcePop masking (their Eqs. 2–3)	$\mathcal{J}_{\text{PPO}} = \sum_t m_t c_t \min\left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 \pm \varepsilon)\hat{A}_t\right)$ (maximized) • **advantage = reward** $\hat{A}_t = \text{sg}\left[\log p_{T_i}(y_t\|s_t) - \log q_{\text{prox}}(y_t\|s_t)\right]$[4] sampled-token RKL log-ratio, in-loss via advantage • **importance ratio** $\rho_t = q_\theta(y_t\|s_t)/q_{\text{prox}}(y_t\|s_t)$ • **behav-to-prox IS weight** $c_t = \text{sg}\left[q_{\text{prox}}(y_t\|s_t)/q_{\text{behav}}(y_t\|s_t)\right]$ • m_t = IcePop token mask $\in \{0, 1\}$ (see "Support mods") • ∇ through live q_θ in ρ_t only	$y \sim q_{\text{behav}}$ (stale; async pipelined; up to 192K tokens)	• PPO clip • IcePop token masking[5] • sg behav-to-prox IS ratio	• same family as the Tinker row: sampled-token RKL advantage, frozen-visitation surrogate — a partial gradient of their own seq-level Eq. 1; the report's "equivalently, at each prefix" bridge is the value identity Eq. 7, not a gradient claim • code-confirmed: `OPDAdvantageEstimator` returns the detached per-token log-ratio; no return-to-go in estimator or loss (Section 4.5)	**S** (med-high — public `ultra-v3` recipe traced; production config unconfirmed)
Nemotron-Cascade 2 MOPD (NVIDIA, 2026a)	MOPD	• per-token RKL on the sampled token as advantage • surrogate stated: REINFORCE with truncated IS correction for train–inference mismatch (their Eqs. 2–4)	$\mathcal{L}_{\text{MOPD}} = -\frac{1}{\|\mathcal{V}(y)\|}\sum_{t\in\mathcal{V}(y)} w_t \ \text{sg}[a_t] \log q(y_t\|s_t)$ • **distillation advantage** $a_t = \log p_{\text{dom}_i}(y_t\|s_t) - \log q(y_t\|s_t)$[6] sampled-token RKL log-ratio, in-loss via advantage • **truncated IS weight** $w_t = \text{sg}[r_t] \cdot \mathbb{1}\left[\varepsilon_{\text{low}} \le r_t \le \varepsilon_{\text{high}}\right]$ (band values in "Support mods") • **train-to-inf importance ratio** $r_t = q(y_t\|s_t)/q_{\text{inf}}(y_t\|s_t)$ • ∇ through q in $\log q(y_t\|s_t)$ only	$y \sim q_{\text{inf}}$ (separate inference engine; T unstated)	• truncated IS band $[0.5, 2]$ • valid-token mask • per-seq valid-token mean	• same family; per-seq mean makes it a length-normalized variant • distillation-only stage (no task reward)	**S** (med — report eqs.; code path not traced)
DeepSeek-V4 OPD (DeepSeek-AI, 2026)	MOPD	• seq-level RKL to N domain experts p_{E_i}, mixed with importance weights w_i (their Eq. 29): $\mathcal{L}_{\text{OPD}} = \sum_{i=1}^{N} w_i \cdot D_{\text{KL}}\left(q_\theta \,\|\, p_{E_i}\right)$ • no surrogate stated — exact full-vocab computation described in prose; the sampled-token sg-advantage reuse of RL frameworks (the Tinker/Nemotron rows) explicitly rejected ("high variance … training instability")	$\mathcal{L} = \sum_{i=1}^{N} w_i \sum_t D_{\text{KL}}\left(q_\theta(\cdot\|s_t) \,\|\, p_{E_i}(\cdot\|s_t)\right)$ full-vocab, exact per position; teacher logits rebuilt from cached last-layer hidden states; TileLang KL kernel • **expert weight** w_i = per-teacher importance weight[7] • token aggregation (sum vs. length-norm mean) unstated • ∇ path unstated: no PG/advantage route; presumed pathwise through live q_θ over the vocab only, none through prefix sampling Where: N = # domain teachers (> 10)	$y \sim q_\theta$ (rollout service; per-domain prompts)	none stated (no clip, IS, or top-K; $\|V\| > 100$k)	• value unbiased for their Eq. 29 (Eq. 7) • presumed-pathwise $\mathbb{E}[\nabla] = \sum_t \mathbb{E}_{s_t}\left[\nabla D_{\text{KL,t}}\right]$ — the Tinker/Nemotron frozen-visitation gradient reached pathwise (GKD-style): sampled-token variance removed, visitation term still omitted	**S** (low-med — prose-level only: ∇ path, aggregation, IS unstated; no code; Section 4.5)
Frameworks: torchtune `ReverseKLLoss`	KD recipe	• tok-level RKL on dataset prefixes (class docstring) • no surrogate	• full-vocab $D_{\text{KL}}(q \,\|\, p)$ per position • ∇ through q; teacher sg	dataset prefixes	-100 label masking	exact token-RKL on dataset visitation	**C**, exists-unused (high)
Frameworks: NeMo-Aligner `bwd_kl`	offline teacher-top-K KD	• tok-level RKL on teacher top-K support (`bwd_kl` option) • truncation inherent to the stored-logits API	• $\sum_{k\in\text{ topK}} \tilde{q}_k(\log \tilde{q}_k - \log \tilde{p}_k)$, both renormalized over teacher top-K ids • hand-written analytic backward at the K ids, zero elsewhere • teacher = dataset constants	dataset tokens (teacher-forced)	teacher-top-K restriction + renormalization **changes which KL**	exact RKL of the K-renormalized conditionals ($\neq$ full RKL)	**C** for restricted objective, exists-unused (high)
k1-in-reward (Ziegler *et al.*, 2019; Ouyang *et al.*, 2022)	RLHF/ RLVR regularizer	• seq-level RKL-penalized reward maximization	$\mathcal{L} = -\sum_t \min(\rho_t A_t, \text{clip}(\rho_t, 1 \pm \varepsilon)A_t) + c_V \mathcal{L}_V$ • **importance ratio** $\rho_t = q_\theta(y_t\|s_t)/q_{\text{old}}(y_t\|s_t)$ (q_{old}: rollout-snapshot policy)	$y \sim q$ (rollout)	• clamps (± 10 OpenRLHF) • adaptive coefs	• idealized: unbiased $-\beta\nabla\text{KL}_{\text{seq}}$ incl. visitation[10] • in practice GAE/critic/clip surrogate	• **C** (high) • NeMo-Aligner $\|k1\|$: **B** as

[3] q_θ: the live student; q_{sample}: the student's log-probs recorded at sampling time (the rollout snapshot) — so ρ_t is the current-vs-sampling ratio, $= 1$ on fresh rollouts.

[4] q_θ: the live student; q_{prox}: the proximal student snapshot serving as the PPO "old" policy; q_{behav}: the stale snapshot that generated the async rollouts; p_{T_i}: domain teacher i.

[5] The report leaves m_t undefined; the `ultra-v3` code defines it — zero out tokens whose IS weight falls outside bounds (reference $[0.5, 5]$). The shipped public recipe (`mopd.yaml`) instead configures a truncated-IS clamp ("tis", $[0.2, 5]$).

[6] q: the training-engine student (π^{train}); q_{inf}: the inference-engine copy of the student that generated the rollouts (train–inference mismatch); p_{dom_i}: the domain teacher selected for the training example; $\mathcal{V}(y)$: the valid response tokens of rollout y retained by the token mask.

[7] p_{E_i}: domain-expert teacher i (an SFT+RL specialist). Each sample is scored by its own domain teacher — samples are dispatched ordered by teacher index, one teacher head in device memory at a time — so the i-sum is realized by data routing.

Occurrence	Stage	Claimed objective & stated surrogate	Expression computed	Sampled from	Support mods	Resulting objective / $\mathbb{E}[\nabla]$	Class.
Frameworks: trl PPO (default) OpenRLHF (default) verl (opt-in) Tülu-3-era PPO (Lambert *et al.*, 2024) NeMo-Aligner PPO ($\lvert k1\rvert$ variant)		• $k1$ in reward is the stated recipe[8]	• **advantage** (GAE over rewards R and critic values V) $A_t = \mathrm{sg}\left[\sum_{k\geq 0}(\gamma\lambda)^k\delta_{t+k}\right], \quad \delta_t = R_t + \gamma V(s_{t+1}) - V(s_t)$ • **per-token reward** $R_t = R_t^{\text{task}} - \beta k1_t, \quad k1_t = \log q_{\text{old}}(y_t\lvert s_t) - \log p(y_t\lvert s_t)$ • R_t^{task} = task/RM reward, typically nonzero only at the final token; V = learned critic (rollout-time values), trained by the value loss $\mathcal{L}_V$ • ∇ through live q_θ in ρ_t only (and through V in $\mathcal{L}_V$) Where: ε = PPO clip; γ, λ = GAE discount and decay (values: footnote in "Resulting objective")		• temperature-scaled logprobs • NeMo-Aligner uses $\lvert k1\rvert$ — a different penalty[9]		named, C for what it implements (high)
k3-as-loss, plain DeepSeekMath GRPO (as written) (Shao *et al.*, 2024) **Frameworks:** verl `low_var_kl` (flagship GRPO recipes) open-instruct (default) trl GRPO (correction off)	RLVR regularizer	• tok-level RKL regularizer • $k3$ stated as an unbiased low-variance **value** estimator, then differentiated as a loss[11]	• $\beta\left(e^{\log p - \log q} - (\log p - \log q) - 1\right)$ per token, in loss (sampled-token) • ref sg, ∇ through $\log q$: $\nabla k3 = (1 - r)\nabla \log q$	$y \sim q$	• clamps: verl/nemo-rl ± 20 on $\log r$, ± 10 on $k3$ • open-instruct ± 40	• $\mathbb{E}[\nabla] = \nabla D_{\mathrm{KL}}(p \,\|\, q)$ **− forward-KL gradient** (Eq. 15) • value estimate remains unbiased for reverse KL	**B** (high; assumes on-policy, clamps not binding)
k3 × differentiable ratio (Tang and Munos, 2025) **Frameworks:** trl GRPO (default) nemo-rl $\exp(x - \mathrm{sg}[x])$ (default) open-instruct estimator 3 (= ratio·k1)	RLVR regularizer	• tok-level RKL regularizer • surrogate stated: bias-corrected $k3$[12]	• $\rho \cdot k3$ with ρ carrying $\nabla \log q$, per token, in loss (sampled-token) • ∇ through both ρ and $k3/k1$	$y \sim q$	as the k3-as-loss row	• on-policy $\mathbb{E}[\nabla] = \nabla D_{\mathrm{KL}}(q \,\|\, p)$ (Eq. 16) • off-policy needs RPG-style weighting; still frozen-visitation per token	**C** on-policy (med-high)
k2-as-loss **Frameworks:** verl "mse"/k2 OpenRLHF k2 verl "k3+" (straight-through)	RLHF/ RLVR regularizer	• tok-level RKL regularizer • $k2$ stated as the low-variance, biased-for-the-value option (Schulman, 2020)	• $(\log q - \log p)^2/2$ per token, in loss (sampled-token)[13] • $\nabla k2 = k1\nabla \log q$	$y \sim q$	clamps	$\mathbb{E}[\nabla] = \nabla D_{\mathrm{KL}}(q \,\|\, p)$ on-policy (Eq. 14)	**C** (high)
Frameworks: trl-classic `"full"`	RLHF regularizer	• seq-level RKL penalty in reward (opt-in `"full"` option) • no estimator — exact per-position KL	$\mathcal{L} = -\sum_t \min(\rho_t A_t, \mathrm{clip}(\rho_t, 1 \pm \varepsilon)A_t) + c_V\mathcal{L}_V$ — the k1-in-reward row's PPO pipeline (ρ_t, GAE A_t, $\mathcal{L}_V$ as defined there), with the penalty derandomized: • **per-token reward** $R_t = R_t^{\text{task}} - \beta \cdot D_{\mathrm{KL}}(q_{\text{old}}(\cdot\,\lvert s_t) \,\|\, p(\cdot\,\lvert s_t))$ exact full-vocab KL, from rollout-snapshot log-softmaxes • R_t^{task} = RM reward at the final token • ∇ through live q_θ in ρ_t only	$y \sim q$ (prefix visitation)	none	• value exact per position, but $\mathbb{E}[\nabla] = -\beta G_{\text{visit}}$ **only**: the penalty is prefix-measurable, so the own-position score term is zero-mean and `no_grad` blocks the pathwise route (Section 4.2) • dual of the token-level pitfall: G_{local} dropped, visitation kept (Section 4.5)	**B**, opt-in (high; value exact)

[8] Sequence-level as written in (Ziegler *et al.*, 2019): $R(x, y) = r(x, y) - \beta \log[\pi(y|x)/\rho(y|x)]$; the per-token placement is stated in (Ouyang *et al.*, 2022): "a per-token KL penalty from the SFT model at each token".
[9] $\mathbb{E}|\log(q/p)|$ is not a KL: it upper-bounds the KL and is symmetric in sign (Section 4.5).
[10] GAE $\gamma = 1.0$ verified at every pinned commit in this row's scope (λ between 0.95 and 1.0), so the reward-to-go carries the visitation term undiscounted.
[11] DeepSeekMath Eq. 4, citing (Schulman, 2020) — where the single-sample estimators are proposed for the KL **value**, "if KL is just being used as a diagnostic".
[12] nemo-rl cites Tang and Munos for the correction factor; trl's `use_bias_correction_kl` (default on) was added after DeepSeek-V3.2.
[13] verl's "k3+" straight-through option returns $k3$'s value with $k2$'s gradient.

4.4 Source table

Table 2 — Sources. One row per distinct training stage / implementation. "Used?" = whether that code path was exercised in the reported experiments (evidence: run scripts, configs, docs). Commits: LMOps `4f2a9de`, trl `24db2a8` (main) / `1f8ba92` (v0.11.4) / `04fd120` (v0.24.0), distillm `d47e77f`, distillm-2 `fe4cf9b`, tinker-cookbook `3d3e976`, torchtune `bd2a0fc`, modelopt `9220fac`, NeMo-Aligner `af2b3bd`, verl `b75f0d0`, OpenRLHF `bc71bb1`, open-instruct `05ad0a2` (and `9502f16` for the deleted Tülu-3-era PPO), nemo-rl `037fb36`, NVIDIA-NeMo/RL branch `ultra-v3` `7314528`.

Report / model	Stage	Claimed objective	Implementation @ commit	Used?	Evidence
MiniLLM (Gu *et al.*, 2024)	KD fine-tune	sequence-level RKL	microsoft/LMOps `minillm` @ `4f2a9de`: `losses.py` L58–196, `sampler.py` L68–131, `reward.py` L38–83; patched sampling in t1101675/transformers @ `be0435e`	**used**	run scripts set `--type minillm --single-step-reg --teacher-mixed-alpha 0.2` etc.
GKD (Agarwal *et al.*, 2024)	KD fine-tune	token-level JSD(β); RKL at $\beta = 1$	huggingface/trl @ `24db2a8`: `experimental/gkd/gkd_trainer.py` L238–323	reference impl.; **paper code not public** (Google-internal, T5)	docs claim fidelity to paper Eq. 1; no official GKD release found in google-research
DistiLLM (Ko *et al.*, 2024)	KD fine-tune	skew SKL / SRKL, adaptive off-policy	jongwooko/distillm @ `d47e77f`: `losses.py` L66–95, `finetune.py` L171–193, L290–325	**used**	scripts set `adaptive-sfkl`/`adaptive-srkl`, `--student-gen`, `kd-ratio 1.0`; one openllama2 script's `srkl-aesop` type matches no code branch (static read)
DistiLLM-2 (Ko *et al.*, 2025)	KD fine-tune	SKL on teacher data + SRKL on student data	jongwooko/distillm-2 @ `fe4cf9b`: `distillm_trainer.py` L1036–1242	**used**	`distillm_v2` config; adaptive-α and gradual-β code present but hard-disabled (L507–509)
TML on-policy distillation (Lu and Thinking Machines Lab, 2025)	post-training	per-token RKL on student rollouts	thinking-machines-lab/tinker-cookbook @ `3d3e976`: `train_on_policy.py` L54–130 loss kernel server-side (docs-verbatim)	**used** (public code for blog); kernel closed-source	recipe defaults `kl_penalty_coef=1.0`, discount `0.0`, `importance_sampling` loss
Nemotron 3 Ultra / Cascade 2 (NVIDIA, 2026b; 2026a)	MOPD	per-token RKL advantage (Ultra: PPO-clipped + IcePop; Cascade 2: truncated-IS REINFORCE)	NVIDIA-NeMo/RL `ultra-v3` @ `7314528`: `advantage_estimator.py` L149–240 (`OPDAdvantageEstimator`), `loss_functions.py` (tis/IcePop modes), `examples/configs/ultra/mopd.yaml`	public recipe ("Nemotron 3 Ultra — MOPD Stage 1"); production run unconfirmed	advantage = `(teacher_logprobs - prev_logprobs).detach()`, no return-to-go in estimator or loss; yaml: `adv_estimator: opd`, "tis" clamp $[0.2, 5]$, `reference_policy_kl_penalty: 0.0`; docstring cites MiMo-V2-Flash Eq. 8 (Xiaomi LLM-Core Team, 2026)
Qwen3 (Qwen Team, 2025)	on-policy distillation	"minimize the KL divergence" on logits (direction unspecified)	none released	—	report text only

Report / model	Stage	Claimed objective	Implementation @ commit	Used?	Evidence
torchtune	KD recipe	forward KL (RKL class shipped)	pytorch/torchtune @ `bd2a0fc`: `kd_losses.py` L11–114	FKL **used** in all shipped configs; `ReverseKLLoss` **exists-unused**	grep over `recipes/configs/`: only `ForwardKLWithChunkedOutputLoss` referenced
NVIDIA modelopt	Minitron-style KD	forward KL	TensorRT-Model-Optimizer @ `9220fac`: `plugins/megatron.py` L292–370	forward **used**; `reverse=True` flag **exists-unused** (unreachable from `DistillationConfig`)	config instantiates `LogitsKLLoss` without exposing `reverse`
NeMo-Aligner KD	offline teacher-top-K KD	`fwd_kl` default; `bwd_kl` = teacher-top-K-renormalized RKL	NeMo-Aligner @ `af2b3bd`: `distributed.py` L510–760 (hand-written backward)	fwd **used** (tutorial); **`bwd_kl` exists-unused**; which path Minitron production used: **indeterminate**	docs/tutorial set `fwd_kl`; no shipped config invokes `bwd_kl`
RLHF classic (trl ≤0.11 PPO)	RLHF regularizer	RKL penalty	trl @ `1f8ba92`: `ppo_trainer.py` L1150–1165	k1-in-reward **default**; "full" (full-vocab RKL) opt-in	config defaults `kl_penalty="kl"`, adaptive coef 0.2
trl ≤0.24 PPO	RLHF regularizer	RKL penalty	trl @ `04fd120`: `ppo_trainer.py` L558–563	k1-in-reward **default**; k3 opt-in	`kl_coef 0.05`; both sides temperature-scaled (default 0.7)
trl GRPOTrainer (main)	RLVR regularizer	RKL penalty (k3)	trl @ `24db2a8`: `grpo_trainer.py` L3094–3146	$\beta = 0$ default (**opt-in**); when on: k3·IS-ratio in-loss (`use_bias_correction_kl=True`)	config cites DeepSeek-R1; bias correction added after DeepSeek-V3.2
verl	RLVR regularizer	RKL penalty	verl @ `b75f0d0`: `core_algos.py` L2187–2251, `losses.py` L131–142	in-loss k3 (`low_var_kl`) set in flagship GRPO recipes; in-reward k1 opt-in; `"full"` raises `NotImplementedError`; `"k3+"` straight-through variant opt-in	`run_qwen3_8b_fsdp.sh` L153–155
OpenRLHF	RLHF regularizer	RKL penalty	OpenRLHF @ `bc71bb1`: `models/utils.py` L64–95	k1-in-reward **default** (coef 0.01); in-loss k1/k2/k3 opt-in	historical flag `--use_kl_estimator_k3` (v0.5.7)
open-instruct (Tülu 3 / OLMo)	RLVR regularizer	RKL penalty	@ `05ad0a2`: `model_utils.py` L793–818, `grpo_utils.py` L504–538; Tülu-3-era PPO @ `9502f16`	grpo_fast in-loss k3 **default** (Tülu 3 recipe $\beta = 0.01$); paper-era PPO used k1-in-reward	recipe `grpo_fast_8b.sh`; deleted PPO file inspected at pinned commit
NeMo-Aligner PPO	RLHF regularizer	RKL penalty	@ `af2b3bd`: `ppo_utils.py` L63–82	$\lvert k1\rvert$-in-reward **default** (`use_absolute_kl: True`)	yaml defaults, coef 0.01
NVIDIA-NeMo/RL (nemo-rl)	RLVR regularizer	RKL penalty	@ `037fb36`: `utils.py` L42–100, `loss_functions.py` L466–521	in-loss k3·$\exp(x - \mathrm{sg}[x])$ **default** (cites Tang & Munos)	`grpo_math_1B.yaml`: k3, coef 0.01
DeepSeekMath GRPO (Shao *et al.*, 2024)	RLVR regularizer	RKL penalty (k3 in loss, Eq. 4)	no official training code	—	paper equations; open frameworks above are the de-facto implementations

4.5 Audit notes and derivations for the non-trivial rows

MiniLLM (S, medium). The claimed objective is sequence-level reverse KL; the shipped estimator is a PPO surrogate whose principal deviations are stated in the paper as deliberate techniques: the exact single-step term (a full-vocabulary per-position RKL, intended in the paper as a variance reduction that **replaces** the sampled own-token term), teacher-mixed sampling (rollouts from $p_{\text{sample}} = 0.8q_{\text{old}} + 0.2p$ against reward hacking), and length normalization (against short-response bias — note this **changes** the objective to a length-normalized variant, which is the point). Unstated deviations: discount $\gamma = 0.95$, advantage whitening, PPO clipping with four reuse epochs, reward scaling and clipping; the retained own-token reward — the paper's decomposition assigns R_{t+1} (excluding r_t) to the policy-gradient part, but `losses.py` builds A_t from the full R_t (`last_rw = rewards[:, t] + gamma * last_rw`) while `--single-step-reg` adds the exact term, so the local gradient component enters twice (once sampled, whitened, and length-normalized; once exact); and — the subtle one — mixture sampling is corrected only by a **per-token** importance weight $w_t = q_{\text{old}}(y_t)/p_{\text{sample}}(y_t)$ (`sampler.py` L110–115), which reweights the action at each step but not the visitation distribution of prefixes (that would require the product of ratios along the prefix). Every listed mechanism is visible in the pinned code. Classification **B** (mild) rather than S: for the PPO composite alone, "correct for a stated surrogate" would apply (its relation to $\nabla\text{KL}_{\text{seq}}$ is not characterized anywhere, only its ingredients), but the single-step handling deviates from the paper's **own stated algorithm** — Algorithm 1 updates with $(\nabla\mathcal{L})_{\text{Single}} + (\nabla\mathcal{L})_{\text{LongNorm}} + \nabla\mathcal{L}_{\text{PT}}$, the long-term part defined with R_{t+1}, i.e. the exact term **replaces** the sampled own-token term, whereas the code retains the full R_t in the advantage and adds the exact term ("single-step regularization" appears nowhere in the paper; the added-regularizer reading exists only in the code's flag name). Unstated deviations from a source's own equations are classified B in this audit (the DistiLLM-2 pattern); mild because the extra term is a second copy of a legitimate gradient component — it vanishes at the optimum and reweights local versus visitation pressure rather than pointing toward a different divergence. Medium confidence.

DistiLLM-2 (B mild, medium). The claimed loss on student data is $D_{\text{SRKL}} = \sum_v q_v(\log q_v - \log m_v)$ with $m = (1-\alpha)p + \alpha q$. Exact differentiation gives $\partial D/\partial q_v = (\log q_v - \log m_v) + 1 - \alpha q_v/m_v$. The code (`distillm_trainer.py` L1194–1198) detaches q inside the mixture, computing $(\log q_v - \log m_v) + 1$ and omitting $-\alpha q_v/m_v$. Since $m_v \geq \alpha q_v$, the omitted term is bounded in $(-1, 0]$; at the optimum $q = p$ it equals $-\alpha$ **uniformly** across the vocabulary, and uniform shifts are annihilated by the softmax Jacobian — so the gradient bias vanishes at the optimum and is bounded elsewhere. The detach appears deliberate (it is asymmetric with the SKL branch, where q inside the mixture stays live) but is not stated in the paper, hence "biased (mildly)" rather than "stated surrogate"; medium confidence.

Tinker (S, high). The blog states the method as per-token reverse KL; the code computes the advantage $A_t = -\text{sg}\big[\log q_{\text{sample}}(y_t) - \log p(y_t)\big]$ and feeds it to a server-side importance-sampling loss $-\sum_t \exp\big(\log q_\theta - \log q_{\text{sample}}\big) A_t$ (kernel formula from official docs; the kernel itself is closed-source — the one indeterminacy). By the derivation in Section 4.2 this is unbiased for the frozen-visitation gradient $\sum_t \mathbb{E}_{s_t}\big[\nabla D_{\text{KL,t}}\big]$ — the same objective GKD computes pathwise over the full vocabulary, estimated here from one sampled token via the score function. It is **not** the full sequence-KL gradient (visitation term omitted; restorable via `kl_discount_factor = 1`, shipped default 0). High confidence: the mechanics are simple and the docs kernel matches the observed inputs.

Nemotron 3 Ultra MOPD — code trace (S, med-high). The `ultra-v3` branch of NVIDIA-NeMo/RL (@ `7314528`) makes the public path traceable: `examples/configs/ultra/mopd.yaml` ("Nemotron 3 Ultra — MOPD Stage 1") selects `adv_estimator: opd`, and `OPDAdvantageEstimator.compute_advantage` returns `(teacher_logprobs - prev_logprobs).detach()` — the per-token advantage of the report's Eq. 2, with no return-to-go anywhere in the estimator or loss (no reverse accumulation exists in either file). This confirms at code level that the update is G_{local} under the behavior policy's visitation: a partial gradient of the report's own sequence-level Eq. 1 in exactly Tang and Munos's sense, nowhere acknowledged as such — the report's bridge sentence "Equivalently, at each prefix s_t …" is the value identity Eq. 7, which holds for the objective value while the gradients differ by G_{visit}. Report–code discrepancies: the shipped recipe configures a truncated-IS clamp ("tis", $[0.2, 5]$) rather than IcePop masking (implemented as an option — zero out tokens with IS weight outside bounds, reference $[0.5, 5]$), and `reference_policy_kl_penalty: 0.0` confirms the advantage is the sole training signal. The estimator's docstring attributes the advantage to "Equation 8 from the MOPD paper" — Xiaomi's MiMo-V2-Flash (Xiaomi LLM-Core Team, 2026), which introduced the MOPD paradigm (explicit reverse-KL loss, gated sampled-token advantage, length-normalized REINFORCE-style update, optional ORM-advantage blending; its Eqs. 5–9). The public recipe is not guaranteed identical to the production run, hence med-high rather than high.

DeepSeek-V4 OPD (S, low-med). The report states the objective at sequence level — Eq. 29, a w_i-weighted sum of policy-level reverse KLs to more than ten domain experts — and the computation at the **integrand** level only: "full-vocabulary logit distillation", with "the exact KL divergences between teacher and student logits … computed using a specialized TileLang kernel" on trajectories sampled from the student (§5.1.2, §5.2.2). No loss-level equations or training code exist, so three estimator-defining facts are unstated: the token aggregation, the gradient path, and any train–inference correction for the rollout service (§5.2.3; sampling temperature and w_i values are also unspecified). The classification presumes the natural reading — pathwise differentiation of the per-position exact KL, no gradient through prefix sampling — under which the value is unbiased for Eq. 29 by Eq. 7 and the expected gradient is the frozen-visitation surrogate $\sum_t \mathbb{E}_{s_t}\left[\nabla D_{\mathrm{KL,t}}\right]$: the same partial gradient as the Tinker and Nemotron rows, reached by the pathwise route instead of the score function. The report's own critique of that family ("high variance in gradient estimation and often causes training instability") concerns the sampled-token **integrand**, which the full-vocabulary computation indeed removes; the omitted visitation term is shared. Alternatives that would change the class: a score-function term restoring G_{visit} (would be C; nothing in the report suggests one), or the advantage-based RL-framework reuse (explicitly rejected). Low-med confidence: prose-level evidence only.

k1-in-reward caveat, NeMo-Aligner PPO. With `use_absolute_kl: True` (the default) the "KL" penalty is $|\log q - \log p|$, whose expectation is **not** the KL divergence (it upper-bounds it and is symmetric in sign); as a named reverse-KL penalty this is inconsistent, as an absolute-log-ratio proximity penalty it is correctly implemented. Flagged because downstream reports citing "a KL penalty" in NeMo-Aligner-era NVIDIA RLHF inherit this substitution silently.

k3-as-loss, the GRPO k3 pattern (B, high). DeepSeekMath's Eq. 4 introduces $k3$ as an "unbiased estimator" — true for the **value** — and then differentiates it as a loss. By Eq. 15 the expected gradient is $\nabla D_{\mathrm{KL}}(p \,\|\, q)$: deployed "reverse-KL-regularized" GRPO in verl (`low_var_kl`) and open-instruct (`kl_estimator 2`) applies, in expectation, **forward**-KL pressure toward the reference. Three independent sources reach the same conclusion (Tang and Munos, 2025; Liu *et al.*, 2025b; Zhang *et al.*, 2025), and two frameworks have since shipped the k3 $\times$ differentiable-ratio repair (TRL's `use_bias_correction_kl`, default on; nemo-rl's surrogate factor, citing Tang and Munos). Confidence high; assumptions: on-policy updates, clamps not binding (verl clamps $\log r$ to ± 20 and $k3$ to ± 10, open-instruct to ± 40 — the clamps censor exactly the tail events where the two directions differ most).

trl-classic `"full"` (B, high). The opt-in `kl_penalty="full"` computes the exact per-position $D_{\mathrm{KL}}(q \,\|\, p)$ over the full vocabulary — `F.kl_div(ref_logprob, logprob, log_target=True).sum(-1)` — and places it in the reward under `no_grad`. Unlike $k1_t$, the penalty does not depend on the sampled token, so the own-position score term has zero expectation and the stop-gradient removes the pathwise term; by the derivation in Section 4.2, the idealized expected gradient of the penalty is $-\beta G_{\text{visit}}$ alone. The option steers rollouts away from high-KL prefixes but exerts, in expectation, no direct pressure on any conditional $q_\theta(\cdot\,|s_t)$ — the mirror image of the token-level pitfall, and a second instance of value-exactness mistaken for gradient-correctness. Verified at the pinned commit: full-vocab branch, `no_grad` reward construction, per-token placement, GAE with $\gamma = 1.0$ (`lam` 0.95, `whiten_rewards` off by default), and the `[:, :-1]` logit slice aligning each penalty with the action drawn from the same prefix. Not covered by Tang and Munos; derived from Eq. 11.

4.6 Support and decoding modifications that change which KL is estimated

Four recurring modifications alter the estimated divergence, usually silently.

- **Temperature.** trl's newer PPO computes both $\log q$ and $\log p$ at the sampling temperature (default 0.7) — the regularizer is the reverse KL between **temperature-scaled** models, not the $T = 1$ models; GKD's trainer conversely generates at $T = 0.9$ but computes the loss at $T = 1$ (legitimate under its any-data-distribution objective, but worth knowing). trl-classic ($\leq$0.11) allowed user generation kwargs to diverge silently from its $T = 1$ scored logprobs.
- **Support truncation and renormalization.** NeMo-Aligner's KD kernel and modelopt's `TopKLogitsKLLoss` renormalize both distributions over the teacher's top-K ids — the result is the exact KL of **different** (conditional) distributions; Gemma 3's pretraining KD does the same with 256 sampled logits ("set to zero probability for non-sampled logits, and renormalized" (Gemma Team, 2025)); nemo-rl's on-policy distillation loss (also teacher top-k: ids from the teacher's `get_topk_logits`, student logprobs gathered at those ids) adds an explicit tail-entropy correction, and its RL path is the only one audited that deliberately computes the KL on **unfiltered** logprobs when rollout top-k/top-p filtering is active.

- **Clamps and clips.** log-ratio and estimator clamps (verl $\pm 20/\pm 10$, OpenRLHF ± 10, open-instruct ± 40, MiniLLM's reward clip ± 100) censor the tails — precisely the region where $q > 0, p \approx 0$ events give reverse KL its zero-forcing character; they also hide the infinities that arise on zero-probability support.
- **Sampling-distribution substitution.** MiniLLM's rollouts come from a teacher-mix, not q, with only a per-token correction (Section 4.3).

5 Reported empirical effects

Diversity. The most careful controlled study, Kirk *et al.* (2024), finds RLHF "substantially decreases the diversity of outputs sampled for a given input compared to SFT" with smaller across-input effects, and — crucially for the direction question — that the reverse-KL knob does not buy the diversity back:

> The results show that this does not work – increasing the KL penalty coefficient leads to a drop in performance as expected, but also to a drop in per-input diversity, rather than a gain.
>
> — (Kirk *et al.*, 2024) · *Understanding the Effects of RLHF on LLM Generalisation and Diversity*

In distribution-matching settings the direction is directly implicated: RKL-DPG shows "low diversity but high quality … suggesting it captures a subset of the target distribution ('mode collapse')" (Go *et al.*, 2023); f-DPO measures lowest predictive entropy for RKL, highest for FKL, JSD between (Wang *et al.*, 2024); BOND reports "the backward KL can lead to policy and entropy collapses" (Sessa *et al.*, 2024). In KD, f-DISTILL quantifies RKL's coverage deficit (Wen *et al.*, 2023), while MiniLLM reports approximately preserved distinct-4 (Gu *et al.*, 2024) and AKL improves quality **and** diversity simultaneously (Wu *et al.*, 2025) — evidence the trade-off is not intrinsic. The 2026 entropy-aware OPD results localize the effect: diversity loss and signal instability concentrate at positions where the teacher distribution has high entropy (Jin *et al.*, 2026).

Mode collapse. Kirk *et al.* (2024) claim the first rigorous demonstration of across-input mode collapse from RLHF specifically; f-DPG's toy-scale observation adds an important qualifier — the entropy crash is transient when the model class can represent the target and severe under misspecification (Go *et al.*, 2023), consistent with the optimization-centric account of Section 2.4.

Calibration. GPT-4's post-training "hurts calibration significantly" (Fig. 8), attributed to post-training as a whole rather than the KL direction (OpenAI, 2023). f-DPO is the source that ties calibration to the constraint choice: "divergence constraints directly influence expected calibration error" (Wang *et al.*, 2024). Against the folklore's expectation, MiniLLM's reverse-KL students were **better** calibrated than its KD baselines (ECE 0.099 on SST-2 vs SeqKD's 0.243) (Gu *et al.*, 2024).

Reward and convergence. RL reward grows roughly linearly in $\sqrt{D_{\mathrm{KL}}(\pi \parallel \pi_{\mathrm{init}})}$ (Bai *et al.*, 2022; Gao, Schulman and Hilton, 2023). As an RL substitute, on-policy reverse-KL distillation converges dramatically faster than RLVR: TML's 7–10× step efficiency and 9–30× cost reduction (Lu and Thinking Machines Lab, 2025), Qwen3's "approximately only 1/10 of the GPU hours" with improved pass@64 (Qwen Team, 2025), and Nemotron-Cascade 2's MOPD converging "in 40–50 steps" while beating GRPO's training efficiency on AIME25 (NVIDIA, 2026a). Frontier adoption followed: DeepSeek-V4 replaced its mixed-RL consolidation stage with pure multi-teacher reverse-KL distillation (DeepSeek-AI, 2026), and Kimi K3 and GLM-5 ship the same sampled-token recipe (Kimi Team, 2026; GLM Team, 2026). On stability, the estimator-level record is mixed: MiniLLM's PG needed three patches; DistiLLM attributes instability to unbounded gradient ratios and fixes it by skewing (Ko *et al.*, 2024); SKD documents garbage teacher feedback on early student samples (Xu *et al.*, 2025); supervision-fidelity decay fades the teacher's corrective signal on long student prefixes (Liu *et al.*, 2026); and thinking-pattern incompatibility can make a stronger teacher useless where a weaker one helps (Li *et al.*, 2026).

Downstream quality. Reverse-KL-family methods win on open-ended instruction following (MiniLLM, GKD's RKL/JSD variants, DistiLLM's SRKL), while the direction is task-dependent elsewhere (GKD; f-DISTILL's MT exception where "mode averaging does not occur") and irrelevant at convergence for representable students (Wu et al.). In production, MOPD gains concentrate exactly where the folklore predicts reverse KL should shine — recovering teacher behaviors on the student's own trajectories (Terminal Bench +16.3; restoration of regressed benchmarks) — with the reported weakness on out-of-support prompts (HLE) attributed by the authors to "a limitation of the on-policy distillation setting rather than a failure of the teacher" (NVIDIA, 2026b). In RLVR, ProRL uses the reverse-KL penalty **for** entropy:

> While DAPO and temperature adjustment help slow entropy collapse, we find that explicit regularization via a KL divergence penalty provides a stronger and more stable solution.
>
> — (Liu *et al.*, 2025a) · *ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models*

The same term that reduces diversity relative to distribution matching preserves it relative to unregularized reward maximization; the two claims are consistent — they differ in baseline.

6 Reproducing the Tinker recipe

The Tinker recipe (Lu and Thinking Machines Lab, 2025) has become the de-facto shared baseline of the 2026 on-policy-distillation literature, but no two papers run it identically. Table 3 collects, per experimental **setting**, the setup needed to reproduce each training run: one column per setting, with closely related settings of the same work sharing a column and distinguished by color (legend in each column header; black values are shared by all of a column's settings). Plain values are stated in the primary source (paper or blog); [code] marks values recoverable only from released code, configs, or SDK defaults; [inf.] marks inferences (arithmetic, framework defaults); "NR" = not reported anywhere examined (paper including appendices, released code, model cards). Quote-level provenance for every cell: `lit_review/notes/repro-settings-dossier.md`.

Two findings deserve flagging before the table. First, the blog's headline "Qwen3-32B → Qwen3-8B-Base" math run carries a footnote reversing its own teacher identity: "We actually use Qwen3-8B as a teacher, as it performs slightly better. However, for the purpose of comparing compute, we can still measure the FLOPs of the 32B model" — the 32B teacher exists only in the FLOPs accounting, and the one paper that faithfully ran a true 32B→8B pipeline (AOPD (Jia *et al.*, 2026)) is reproducing a configuration the blog itself did not run. Second, the evaluation protocol behind the blog's 60%/70% AIME'24 numbers was never published: no sample count, metric, temperature, or grader at blog time, with only a partial disclosure in the June 2026 cookbook README (for the Qwen3.5 rerun: temperature 1.0, top-p 1.0, top-k −1, max_tokens 64000 — samples per problem still unstated). Of the eight works tabulated, four (TML, OPD+ (Zhao *et al.*, 2026), AOPD, Geometry (Shen *et al.*, 2026)) report no complete evaluation protocol, and Geometry reports no numeric accuracy at all (training curves only).

Table 3 — Reproduction setups for Tinker-recipe-family settings. Colors identify settings within a column (header legend). NR = not reported; [code] = from released code/configs/SDK defaults, not the paper; [inf.] = inferred. Sub-variants (SFT-warm-up length, teacher sweeps) noted inline. "→" separates result progressions (baseline → method arm).

	TML T1 — blog headline (full FT) (Lu and Thinking Machines Lab, 2025)	**TML T2 — cookbook (LoRA-128)** (Lu and Thinking Machines Lab, 2025)	**OPD+** (Zhao *et al.*, 2026) B1 math · B2 tool-use	**AOPD** (Jia *et al.*, 2026) A1 32B→8B · A2 8B→4B	**Geometry of OPD** (Shen *et al.*, 2026) G1 8B←32B · G3 4B, teacher sweep · G4 14B	**SCOPE** (Zheng *et al.*, 2026) S1 1.3B math · S2 Qwen3-1.7B math	**EOPD** (Jin *et al.*, 2026) E1 0.6B · E2 1.7B · E3 4B	**LGR** (Lin *et al.*, 2026) L1 math 1.5B · L3 math 7B	**Rethinking OPD** (Li *et al.*, 2026) R1–R2 · R3 cold-start · R4–R5 · R6–R8 · R9–R11
Identity									
teacher	Qwen3-32B (as presented) blog footnote: "We actually use Qwen3-8B as a teacher, as it performs slightly better" – 32B enters only the FLOPs accounting	Qwen3-8B (script default; README passes none)	Qwen3-8B	Qwen3-32B Qwen3-8B	Qwen3-32B; sub-var.: Qwen3-30B-A3B Qwen3-8B / 14B / 32B Qwen3-32B	Skywork-OR1-7B "Qwen3-8B-Instruct" as printed — no such release; repo uses Qwen3-8B [code] think-mode NR	Qwen3-8B, non-thinking (chat template pre-fills empty `<think>` block)	Skywork-OR1-Math-7B R1-Distill-Qwen-32B	R1: Qwen3-4B (NT) · R2: Qwen3-4B-Base-GRPO (author-trained, HF) R3: Qwen3-4B (NT) R4: Qwen3-4B (NT) · R5: Qwen3-4B-NT-RL-Math (external, arXiv 2602.12125) R6: R1-D-7B · R7: Skywork-OR1-Math-7B · R8: JustRL-1.5B R9: R1-D-1.5B · R10: R1-D-7B · R11: Skywork-OR1-Math-7B / R1-D-14B
student	Qwen3-8B-Base	Qwen3-8B-Base (LoRA rank 128)	Qwen3-8B-Base	Qwen3-4B-Base Qwen3-4B-Base	Qwen3-4B Qwen3-4B Qwen3-14B	R1-Distill-Qwen-1.5B Qwen3-1.7B-Base	Qwen3-0.6B-Base Qwen3-1.7B-Base Qwen3-4B-Base	R1-Distill-Qwen-1.5B R1-Distill-Qwen-7B	Qwen3-1.7B-Base Qwen3-1.7B-SFT (control: -Base) Qwen3-1.7B (NT, instruct) R1-Distill-Qwen-1.5B JustRL-1.5B (R9–10); R1-Distill-Qwen-7B (R11)
Meta									
code available	partial — tinker-cookbook @ `330b73d8` (LoRA-on-Tinker path only; full-FT run was internal)	yes — tinker-cookbook @ `330b73d8` (README commands)	no ("one line change" never linked)	no	no	yes — `machine9801/SCOPE` (verl fork) ⚠ skipped scripts deviate from the paper's equations — see loss row	yes — `WLS04/EOPD` (verl snapshot; referenced run script missing)	partial — `zui-jiang/LGR` (SLIME fork; only the 1.5B LGR-topk script; drifts: student:teacher topK 20:20 vs 10:10, step cap 3000 vs 300)	yes — `thunlp/OPD` (verl v0.7.0; SFT via LLaMA-Factory v0.9.5)
Evaluation on AIME'24									
n_seeds	NR	NR	NR (1 run/arm inf.)	NR	NR	NR	NR	1 run/arm (checklist: no error bars)	NR (eval seed = sample index [code])
n_samples / metric	NR	NR	avg@32, best checkpoint	Pass@1 & Pass@4; 16 samples/problem [inf. from score grid]	avg@8 ⚠ no numeric accuracy anywhere in the paper — curves only	Avg@32 + Pass@32 (repo Pass = bootstrap, 1000 resamples [code])	Avg@8 + Pass@8 (8 samples; Pass@k curves to k=128)	mean@8 + pass@8	avg@16
prompting	NR	NR	NR	NR	NR	question + boxed tip, single user turn, no system prompt [code] (literal `\boxed{{}}` bug)	zero-shot; "Please reason step by step, and put your final answer within `\boxed{}`" (Qwen2.5-Math pipeline)	NR (chat template [code])	JustRL-style boxed prompt [code] — differs from training template
answer extraction & grading	NR	NR	NR	NR	NR	math_robust / math_verify / prime_math per benchmark [code]	Qwen2.5-Math pipeline (boxed extraction + rule grader)	DeepScaleR-style boxed verify [code]	math-verify (JustRL pipeline) [code]
stop condition	NR	NR	NR	NR	NR	NR	EOS / max [code]	NR (pipeline default)	NR
temperature	NR (June-2026 rerun README: 1.0)	NR (rerun: 1.0)	NR	NR	NR	0.6	1.0	1	0.7
top_p	NR (rerun: 1.0)	NR (rerun: 1.0)	NR	NR	NR	0.95	0.8	1 [code]	0.95
top_k	NR (rerun: −1)	NR (rerun: −1)	NR	NR	NR	20	NR	NR	NR
max_tokens	NR (rerun: 64000)	NR (rerun: 64000)	NR	NR	NR	32768	8192	39k (39,936 [code])	31,744
other benchmarks (same protocol)	GPQA-D in blog table = Qwen3-report values, not TML measurements	—	AIME'25, HMMT'25; ToolAlpaca 68-problem split	AIME'25, HMMT'25-Feb; ToolAlpaca success rate (continual stage)	"five reasoning benchmarks" named only in figure images; LCB v5 (code track)	MATH500, AIME'25, AMC'23, Minerva, OlympiadBench; repetition penalty 1.08 (eval-time; "severe repetition issues") 1.0	MATH500, AIME'25, AMC'23, Minerva, OlympiadBench; OOD (E2): GPQA-D, MMLU-Pro, AlpacaEval 2.0	AIME-25/26, HMMT-25/26	AIME'25, AMC'23
SFT stage									
student (init)	Qwen3-8B-Base	Qwen3-8B-Base + LoRA-128	Qwen3-8B-Base ("initialized by SFT")	Qwen3-8B-Base Qwen3-4B-Base	Qwen3-8B-Base (authors' anchor) "Qwen3-4B/14B-SFT" — provenance NR	— no SFT stage (released ckpt / raw base)	— no SFT stage (raw base students)	— no SFT stage (released R1-Distill ckpts)	R3 only: Qwen3-1.7B-Base
dataset	OpenThoughts3-1.2M, "400k prompts" (= 128·3000 = 384k [code])	OpenThoughts3-1.2M, 384k prompts, shuffle buffer 384k [code]	NR (never named)	"OpenThoughts" (version/size/provenance NR)	Dolci-Think SFT	—	—	—	206K Qwen3-4B (NT) rollouts on OpenThoughts3 math prompts (T 0.7, top-p 0.95, max 12,288; truncated/degenerate filtered); released on HF
num_steps	3000 [inf.]	3000	NR	3000 ("strong") / 1000 ("weak") sub-variants	5 epochs = 5375 steps [inf. from ckpt name]	—	—	—	1 epoch (≈3.1k steps [inf.])
batch_size	128 [code]	128	NR	128	256	—	—	—	64 (8 GPU × 8)
lora_rank	false (full FT)	128	NR (Tinker ⇒ LoRA implied; rank unstated)	NR (full FT inf.)	false [inf.]	—	—	—	false (full-param stated)
max_length	16384 [code]	16384 [code]	NR	NR	NR	—	—	—	14,336 (cutoff)
loss mask	all assistant messages [code]	all assistant messages [code]	NR	NR	"Qwen3 loss mask" (entire statement)	NR	—	—	NR
renderer / template	`qwen3` [code]	`qwen3`	NR	NR	NR	—	—	—	`qwen3` (LLaMA-Factory)
learning_rate	1e-4 (README), linear → 0	1e-3, linear → 0	NR	NR	1e-5 cosine → 1e-6, warmup 10%	—	—	—	1e-5 cosine, warmup 0.05
optimizer & settings	Adam(β 0.9/0.95, ε 1e-8); no weight decay or grad clip (absent from blog-era SDK); 1 epoch [code]	same as T1	NR	NR	Adam, wd 0.1, β (0.9, 0.95)	—	—	—	BF16; ZeRO-2, FlashAttention-2, Liger [code]
post-SFT ckpts available	no	yes — Tinker-hosted (Dec 2025), ranks 8/32/128, `weights/final`	no	no	no	—	—	—	yes — HF `tlyx/Qwen3-1.7B-SFT`
post-SFT AIME'24	60%	≈55% (after 3000 steps)	50.94 (avg@32)	Pi@1: 58.96 (3K) / 45.21 (1K) · 52.29 (3K) / 36.25 (1K)	NR (figure only)	—	—	—	figure-only (Fig. 8 step-0)
other post-SFT results	rank-32 LoRA trails full FT by 13% after SFT (blog)	—	AIME'25 38.61, HMMT'25 25.52, ToolUse 24.63	AIME'25/HMMT'25 per Table 1/3	—	—	—	—	—
OPD stage									
teacher	see Identity (footnote: actual Qwen3-8B)	Qwen3-8B	Qwen3-8B	Qwen3-32B Qwen3-8B continual tool stage: NR	Qwen3-32B (sub-var. 30B-A3B) 8B/14B/32B 32B	Skywork-OR1-7B Qwen3-8B	Qwen3-8B (non-thinking)	Skywork-OR1-Math-7B R1-D-32B	see Identity
student init	400k full-FT SFT checkpoint	rank-128 SFT checkpoint (`load_checkpoint_path`)	SFT checkpoint (data unnamed)	SFT warm-up checkpoint (5K or 1K)	SFT anchor `iter_0005375` "Qwen3-4B/14B-SFT"	released ckpt raw base	raw base	released ckpts	base SFT ckpt (control: base) instruct released ckpts
on-policy sync	fully synchronous; sampler refreshed after **every** optimizer step (1-step lag corrected by IS ratio); `num_substeps`= 1	same as T1	NR (Tinker loop implied)	NR (on-policy implied)	on-policy, student-generated; refresh cadence NR	1 update per rollout batch (256 = 256 mini), $\mu \equiv 1$; old logprobs recomputed on the trainer	PPO-style: B=128 prompts, mini 32 ⇒ 4 grad steps per rollout iteration	1 step per fresh rollout batch, staleness 0 [code]	1 update/batch, ratio $\equiv 1$ [code]
dataset	DeepMath-103K (README; blog never names it)	DeepMath-103K, `question` field	DeepMath - ToolAlpaca (subset/size NR)	DeepMath (split/filtering NR)	dapo-math-17k	DeepMath (split NR repo: `sample_deepmath_train.json`, ≤16,384-char filter)	MATH (repo: MATH-lighteval train, 7.5k [inf.]) "DAPO-Math-14k" = DAPO-Math-17k Processed "en", 14,116 rows [code]	Polaris (repo pins 53K version)	DAPO-Math-17K; R3: +30K held-out OpenThoughts3 math prompts
num_steps	≈150 (≈77K prompts)	100 (README score point; full epoch ≈202 [inf.])	100 - 80	90 (≈40 ToolAlpaca continual, 3K-arms only)	299 (endpoint at eval plateau)	NR (repo: 10 epochs)	3 epochs ≈176 iters [inf.] 2 epochs ≈220 [inf.]	300	1 epoch ≈270 [inf.] (length-sweep arms: 200)
data ordering	sequential, no shuffle [code]	sequential, no shuffle [code]	NR	NR	NR	NR (verl shuffle default inf.)	NR (repo shuffle: true)	shuffled [code]	sequential (`shuffle=False`) [code]
rollouts_per_group	4	4	NR (256 rollouts/step total)	NR	8	1 (OPD baseline; SCOPE arm: 8)	1 (GRPO arm: 8)	1	4
groups_per_batch	=512 [arith.: 77K/150; README: 512]	512	64 "groups" - NR	512 ("batch size 512"; prompts vs sequences ambiguous)	300 prompts/step; "global batch size 64" (tension unexplained)	256	128 (mini 32)	128	64
prompt construction	no system prompt; raw question; truncated to 1024 tok [code]	same as T1	NR	NR	NR	question + boxed tip, no system prompt [code]	single user msg + boxed instruction [code] teacher template pre-fills empty `<think>`	chat template; max prompt 1024 [code]	DAPO template ("Solve the following math problem step by step...Answer:"); max prompt 1024; NT students need `enable_thinking=False`
rollout temperature	1.0 [SDK default — nothing plumbed]	1.0 [SDK default]	NR	NR	1.0	0.6	1.0	1.0 (student **and** teacher)	1.0
rollout top_p	1.0 [SDK]	1.0 [SDK]	NR	NR	NR (verl 1.0 inf.)	NR (verl 1.0 inf.)	1.0	NR (1.0 inf.)	1.0
rollout top_k	−1 [SDK]	−1 [SDK]	NR	NR	NR	NR (-1 inf.)	NR (-1 inf.)	NR	NR (-1 inf.)
rollout max_tokens	4096 [code]	4096 [code]	NR	NR	NR	12,288 (prompt cap 4096; ⚠ repo scripts 2048)	4096	9k main arm (sweep 3k/9k/16k/39k)	7168
rollout stop condition	renderer stop sequence ⚠ blog-day default resolves Qwen3-8B-Base ⇒ `role_colon` ("\nUser:"); intended `qwen3` (`im_end`=5). fixed later on main	same caveat as T1	NR	NR	NR	EOS/length [inf.]	NR	NR	EOS [inf.]
framework	Tinker (server-side loss kernel)	Tinker	Tinker [implied, not stated]	NR	NR (Megatron-style ckpt names; sglang teacher serving)	verl (FSDP + vLLM) [code]	verl (KD arm: DistillKit)	SLIME + SGLang (Megatron backend)	verl v0.7.0 [code]
learning_rate	5e-5, constant (README full-FT)	1e-4, constant	NR	1e-5	1e-6, constant	5e-5, constant; wd 0.01	5e-5, cosine	3e-6, cosine	1e-6, constant, no warmup [code]
lora_rank	false	128	NR (LoRA implied via Tinker)	NR (full inf.)	false [inf.]	false (FSDP full FT)	false [inf.]	false [inf.]	false (full-param FSDP)
advantage	$A_i = -\mathrm{sg}\big[\log q_{\mathrm{samp}}(y_i) - \log p_T(y_i)\big]$; coef 1.0; discount 0 zero rewards + 8-group-centering annihilates them	same as T1	$A_i = \mathrm{sg}[\log p_T(y_i) - \log p_\theta(y_i)]$ OPD+ arm: $u_i > -1$ (Table 1: §3.2 prose sign-flipped)	$A_t = \mathrm{sg}[\log p_T(y_t) - \log p_\theta(y_t)]$	$A_t = -(\log \pi_\theta(y_t) - \log \pi_T(y_t))$, sampled-token Table 4 says "GRPO estimator", App. G.2 formula is the raw gap, normalisation ambiguous	$\mathrm{sg}[\log \pi_\theta(y_t) - \log \pi_T(y_t)]$ as negative advantage, wrong-answer branch only ⚠ repo instead ships −k3: $-(\exp(\epsilon^k) - \Delta - 1, \pm 10)$, $\Delta = \log \pi_T - \log \pi_{\text{old}}$	$\hat{A}_t = \log \pi_{\text{sg}}(x_t) - \log \pi_{\theta_{\text{old}}}(x_t)$	$A_t = 1 + \log \pi_\theta(x_t) - \log \pi_T(x_t)$ note the +1	per $(t, v \in \text{student top-16})$: $-\mathrm{sg}[\log p_\theta(v) - \log q_\theta(v)] \cdot \hat{p}_\theta(v)$ [code]
loss	$L = -\sum_y \exp(\log q_\theta(y_i) - \log q_{\mathrm{samp}}(y_i)) \cdot A_i$ – unclipped IS, server-side kernel	same as T1	PPO-like IS surrogate $-\mathbb{E}\sum_a \rho_a(\theta) A_a$, $\rho_a = p_\theta/p_{\theta_{\text{old}}}$; no clipping	$L = -\mathbb{E}\big[(1/\lvert y\rvert)\sum_t A_t \log p_\theta(y_t)\big]$ – length-normalized; no IS ratio, no clipping AOPD arm: tokens with $A_t \le 0$ get truncated FKL on teacher top-K = 32	policy gradient on A_t; clip ε NR (off-policy arm: ε=1, inactive)	$\sum_t \rho_{i,t}(\log \pi_\theta - \log \pi_T)$ (wrong) + PPL-weighted $-\sum_t \rho_{i,t}$ MLE (correct); no clipping in paper	PPO-clipped: $\max\{-r_t \hat{A}_t, -\mathrm{clip}(r_t, 1 \pm \varepsilon)\hat{A}_t\}$; ε NR (verl 0.2 inf.) EOPD arm: $\star$ α1[$H^* > 0.8$] · FKL on teacher top-16, α=1	$\nabla = \sum_t \nabla \log \pi_\theta \cdot A_t$ (stop-grad RKL) topk arm: student π teacher top-10, renormalized (⚠ repo: 20/20 + PPO-clip).	student-top-16 renormalized RKL: $\mathbb{E}\sum_t D_{\mathrm{KL}}(\tilde{p}_\theta^{S_K} \,\|\, \tilde{q}_\theta^{S_K})$, realized as PG with ratio = 1 [code] sampled-token TML estimator + §6.3 ablation arm only
loss mask	sampled completion tokens	sampled completion tokens	NR	NR	response tokens [inf.]	response tokens; token-mean [code]	response tokens; token-mean	NR (response inf.)	response tokens; token-mean
optimizer & settings	Adam(β 0.9/0.95, ε 1e-8); no wd, no clip [code+SDK]	same as T1	NR	NR	Adam, wd 0.1, β (0.9, 0.98); ε NR	NR (repo AdamW (β 0.9, 0.999), warmup 0)	AdamW (β/wd NR)	AdamW (verl defaults inf.)	AdamW (verl defaults inf.)
post-OPD ckpts available	no	yes — Tinker-hosted (Dec 2025), ranks 8/32/128, `sampler_weights/000080` (step 80)	no	no	no	SCOPE arms only (HF); no OPD/GRPO/KD arms	no	no	no student ckpts (teacher GRPO ckpt + SFT corpus released)
post-OPD AIME'24	70% (+150 steps)	≈65% (100 steps)	OPD 64.27 → OPD+ 64.69 (avg@32)	OPD: 61.46 (3K) / 43.13 (1K, **below** SFT) · 57.08 / 46.25 AOPD: 66.87 / 53.75 · 61.04 / 51.25 (Pi@1)	NR — figures only	(Avg@32) base 29.4 → OPD 40.2 → SCOPE 42.7 base 3.7 → OPD 12.2 → SCOPE 13.3	(Avg@8) OPD: 2.50 8.33 18.33 EOPD: 4.17 10.42 17.92	(mean@8) base 24.17 → OPD(9k) 45.83 → LGR 46.67 base 52.08 → OPD(9k) 62.08 → LGR 61.67	figure-only. Text: R8 recovers ≈80% of teacher gap; R6 "no improvement" R9/R10 regress to pre-RL level
other post-OPD results	9–30× compute vs extrapolated SFT; ≈18× GPU-hours (blog)	—	ToolUse: OPD 59.97 → OPD+ 61.44; FKL/JSD OPD collapse to 0.00	tool-use success after continual stage: OPD 65.0, AOPD 75.0, GKD 76.0 (1K arms)	bf16 update sparsity (Table 1): 51.6% (MoE-teacher 48.6%) 50.3/51.1/51.7% 56.6%	6-bench avg: OPD 52.3 → SCOPE 55.2 OPD 30.6 → SCOPE 32.5	6-bench deltas EOPD–OPD: +1.16/+0.99/+1.80 Avg@8	AIME-25/26, HMMT-25/26 rows in Table 1; length sweep Table 2	teacher-continuation probe: teacher advantage +0.37 @1K prefix → +0.02 @16K

Reading the table columnwise, the recipe's reproducibility rests on three uneven legs. The **algorithm** is the well-specified part: every column implements a sampled-token student–teacher log-ratio as a detached per-token signal, and the differences (the +1 baseline in LGR, OPD+'s $\ln u - 1$ correction, EOPD's PPO clip, Rethinking's student-top-16 support, AOPD's length normalization, the presence or absence of an importance ratio) are all visible in the loss row. The **pipeline** is bimodal: TML, OPD+, AOPD, Geometry (G1), and Rethinking's R3 run a true SFT→OPD staging, while every other setting outsources or skips the SFT stage — and where the SFT stage exists, it is the worst-documented part (OPD+ never names its SFT data; Geometry's 4B/14B anchors have no stated provenance). The **evaluation** is the weakest leg: only SCOPE, EOPD, LGR, and Rethinking publish a complete sampling protocol, no work reports eval seeds, and the two Tinker-defaults reruns (T2, OPD+ B1) land at 64–65% AIME'24 while the blog-day headline's 70% rests on an unpublished protocol and a teacher that, per the blog's own footnote, was not the 32B model the headline advertises.

7 Criticism and alternatives

Criticism of the divergence itself. (i) Mode/coverage loss: acknowledged by MiniLLM, quantified by f-DISTILL and GKD, measured in alignment by Kirk, f-DPG, f-DPO, BOND (Section 5), and localized to high-entropy teacher positions at 2026 scale (Jin *et al.*, 2026). (ii) The mode-seeking justification may be vacuous at LLM scale: softmax students share the FKL/RKL optimum; observed differences are early-training head-versus-tail dynamics (Wu *et al.*, 2025). (iii) Over-concentration: ABKD frames RKL as excessive "confidence-concentration" (Wang *et al.*, 2025), and DRKL reports RKL's non-target gradients "push the target logit upward even when the student already matches the teacher" (Luong, Tran and Chen, 2026). (iv) In its regularizer role, the penalty does not improve the gold-reward frontier (Gao, Schulman and Hilton, 2023) and its coefficient does not trade back diversity (Kirk *et al.*, 2024).

Criticism of estimators and implementations — a separate matter. The naive sequence-level objective admits degenerate optima requiring patches (MiniLLM's three techniques); pure on-policy data yields unreliable teacher signals early (SKD) and off-policy reuse breaks vanilla directions (DistiLLM); the k3-as-loss pattern optimizes the wrong direction in expectation (Section 4.5); token-level losses drop the visitation term (Tang and Munos) while exact-KL-in-reward drops the local term (Section 4.5); off-policy updates need importance-weight corrections that GRPO's KL term lacks (RPG); clamps censor exactly the zero-forcing tail; the 2026 OPD studies add data-level failure modes — the teacher's corrective signal fades on long on-policy prefixes (Liu *et al.*, 2026), teacher–student thinking-pattern incompatibility bounds what distillation can transfer (Li *et al.*, 2026), and the agreement signal itself is gameable by student-manufactured contexts: repetition-induced degenerate agreement (Yu *et al.*, 2026) and low-KL agreement traps (Xin *et al.*, 2026) (Section 2.4). Nemotron 3 Ultra's failure report (§Section 3.3) belongs here, not with the divergence-level criticism: the reverse KL stayed, the full-distribution estimator went.

Divergences introduced to fix FKL/RKL shortcomings. The interpolation move recurs at every scale:

- **Minka's α-family** — "The divergences with $0 < \alpha < 1$ are a blend of these extremes" (Minka, 2005).
- **Generalized JS** — Huszár's JS_π interpolation (Huszár, 2015).
- **JSD(β)** — GKD's forward-to-reverse knob (Agarwal *et al.*, 2024).
- **JS and TVD** — f-DISTILL's symmetric losses ("symmetric … consistently better" (Wen *et al.*, 2023)).
- **Skew KL/RKL** — DistiLLM's mixtures, for gradient stability rather than mode behavior (Ko *et al.*, 2024).
- **AKL** — adaptive head/tail weighting (Wu *et al.*, 2025).
- **α-β family** — ABKD (Wang *et al.*, 2025).
- **Jeffreys divergence** — BOND, chosen with the both-failure-modes argument quoted in Section 2.3 ("the Jeffreys divergence inherits the best of both divergences" (Sessa *et al.*, 2024)).
- **f-divergence constraint families** — f-DPO and f-PO (Wang *et al.*, 2024; Han *et al.*, 2025).
- **Forward-KL distribution matching** — the mirror-image fix (GDC/DPG (Khalifa, Elsahar and Dymetman, 2021), f-DPG's KL-DPG — itself penalized by high-variance importance weights (Go *et al.*, 2023)). A unified theory of f-divergence-regularized RLHF is emerging (Wu *et al.*, 2026). Notably, DistiLLM-2's result that **same-data** interpolation does not help — only direction-matched-to-data pairing does (Ko *et al.*, 2025) — suggests the useful axis is often data-conditional rather than a global β.

8 Synthesis

On the folklore (Q1–Q2). The when-to-use-which story is theoretically grounded but rests on capacity limitation and global optimization, assumptions that the LLM setting often violates; its standard figures all depict a unimodal

family fit to a mixture. Empirical support at LLM scale is real but narrower than the folklore: reverse-KL-family objectives win on open-ended generation quality under sampling, with measurable diversity costs; the differences shrink or vanish with student capacity, greedy decoding, or training time. The strongest criticisms relocate mode collapse from the objective to optimization dynamics and misspecification.

On usage (Q3, Q5). Reverse KL earns its keep in exactly two production roles: the universal-but-small RLHF/RLVR regularizer (where it is the **tractable** direction, and where the variational view makes it principled), and on-policy post-training distillation (where it doubles as a dense, unhackable reward and an order-of-magnitude-cheaper RL substitute — by 2026 the standard frontier consolidation stage: DeepSeek-V4, Kimi K3, GLM-5, Nemotron). The regularizer role, meanwhile, has become contested: one prominent RLVR line removes the term outright while another keeps and repairs its estimator (Section 3.2). No production pretraining-scale distillation uses it, and the two clean production negatives bracket the regime: full-vocabulary reverse KL failed both off-policy at scale (Minitron) and on-policy at the estimator level (Nemotron 3 Ultra §3.3.5).

On implementations (Q6). The audit's central finding is that the field's stated objectives and computed gradients frequently diverge, in patterned ways: value-unbiasedness is routinely mistaken for gradient-correctness (k3-as-loss, expected gradient in the **forward** direction — now being repaired in TRL and nemo-rl; trl-classic's exact-KL-in-reward `"full"` option, expected gradient G_{visit}-only); token-level estimators silently optimize frozen-visitation objectives; support truncations, renormalizations, and clamps change which divergence is estimated; and several reverse-KL code paths (torchtune, NeMo-Aligner `bwd_kl`, modelopt `reverse`) exist without any shipped experiment using them. The successful production estimator — sampled-token log-ratio as a per-token advantage — is best understood not as "reverse KL, approximated" but as a deliberately partial, per-token surrogate whose robustness (only realized actions are supervised; off-support teacher noise is never queried) is plausibly **why** it beat the "more correct" full-distribution estimator.

Open questions. When does the visitation-gradient term matter in practice (discount 0 versus 1 in on-policy distillation)? Is the Nemotron full-distribution failure about teacher miscalibration off-support (their hypothesis) or about the gradient structure of full-vocabulary RKL on student prefixes? And does the direction question survive controlled comparisons at production scale — on-policy **forward** KL at sampled tokens being the natural missing baseline in nearly every comparison reviewed here?

Raw dossiers with full verbatim quotes, per-source BibTeX, and additional audited code paths: `lit_review/notes/A–F.md`, plus per-paper notes (`notes/per-paper/`) and the unincorporated-papers dossier (`G-unincorporated.md`). Quotes were verified against primary sources; the Schulman blog was reachable only via a rendering proxy and its formulas were cross-checked against secondary sources. MiniLLM quotes follow the current arXiv v6 and may differ slightly from the ICLR 2024 camera-ready. The original figures reproduced in Section 2.2 are © their respective rightsholders and are included in this unpublished, personal-use document for private study and commentary only.

Bibliography

Agarwal, R. *et al.* (2024) "On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes," in *International Conference on Learning Representations*.

Amini, A. *et al.* (2024) "Variational Best-of-N Alignment," *arXiv preprint arXiv:2407.06057* [Preprint].

Apple (2024) "Apple Intelligence Foundation Language Models," *arXiv preprint arXiv:2407.21075* [Preprint].

Arjovsky, M. and Bottou, L. (2017) "Towards Principled Methods for Training Generative Adversarial Networks," in *International Conference on Learning Representations*.

Bai, Y. *et al.* (2022) "Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback," *arXiv preprint arXiv:2204.05862* [Preprint].

Bishop, C.M. (2006) *Pattern Recognition and Machine Learning*. Springer.

Blei, D.M., Kucukelbir, A. and McAuliffe, J.D. (2017) "Variational Inference: A Review for Statisticians," *Journal of the American Statistical Association*, 112(518), pp. 859–877.

Chan, A. *et al.* (2022) "Greedification Operators for Policy Optimization: Investigating Forward and Reverse KL Divergences," *Journal of Machine Learning Research*, 23(253), pp. 1–79.

DeepSeek-AI (2025) “DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models,” *arXiv preprint arXiv:2512.02556* [Preprint].

DeepSeek-AI (2026) “DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence,” *arXiv preprint arXiv:2606.19348* [Preprint].

DeepSeek-AI, Guo, D. and others (2025) “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” *arXiv preprint arXiv:2501.12948* [Preprint].

Gao, L., Schulman, J. and Hilton, J. (2023) “Scaling Laws for Reward Model Overoptimization,” in *International Conference on Machine Learning*.

Gemini Team (2024) “Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context,” *arXiv preprint arXiv:2403.05530* [Preprint].

Gemma Team (2024) “Gemma 2: Improving Open Language Models at a Practical Size,” *arXiv preprint arXiv:2408.00118* [Preprint].

Gemma Team (2025) “Gemma 3 Technical Report,” *arXiv preprint arXiv:2503.19786* [Preprint].

GLM Team (2026) “GLM-5: From Vibe Coding to Agentic Engineering,” *arXiv preprint arXiv:2602.15763* [Preprint].

Go, D. *et al.* (2023) “Aligning Language Models with Preferences through f-divergence Minimization,” in *International Conference on Machine Learning*.

Goodfellow, I. (2017) “NIPS 2016 Tutorial: Generative Adversarial Networks,” *arXiv preprint arXiv:1701.00160* [Preprint].

Gu, Y. *et al.* (2024) “MiniLLM: Knowledge Distillation of Large Language Models,” in *International Conference on Learning Representations*.

GX-Chen, A. *et al.* (2025) “KL-Regularized Reinforcement Learning is Designed to Mode Collapse,” *arXiv preprint arXiv:2510.20817* [Preprint].

Han, J. *et al.* (2025) “f-PO: Generalizing Preference Optimization with f-divergence Minimization,” in *International Conference on Artificial Intelligence and Statistics*.

Huszár, F. (2015) “How (not) to Train your Generative Model: Scheduled Sampling, Likelihood, Adversary?,” *arXiv preprint arXiv:1511.05101* [Preprint].

Jang, E. (2016) *A Beginner's Guide to Variational Methods: Mean-Field Approximation*. Available at: https://blog.evjang.com/2016/08/variational-bayes.html.

janus (2022) *Mysteries of Mode Collapse*. Available at: https://www.lesswrong.com/posts/t9svvNPNmFf5Qa3TA/mysteries-of-mode-collapse.

Ji, H. *et al.* (2024) “Towards Efficient Exact Optimization of Language Model Alignment,” in *International Conference on Machine Learning*.

Jia, N. *et al.* (2026) “Asymmetric On-Policy Distillation: Bridging Exploitation and Imitation at the Token Level,” *arXiv preprint arXiv:2605.06387* [Preprint].

Jin, W. *et al.* (2026) “Entropy-Aware On-Policy Distillation of Language Models,” in *International Conference on Machine Learning*.

Khalifa, M., Elsahar, H. and Dymetman, M. (2021) “A Distributional Approach to Controlled Text Generation,” in *International Conference on Learning Representations*.

Kim, G., Jang, D. and Yang, E. (2024) “PromptKD: Distilling Student-Friendly Knowledge for Generative Language Models via Prompt Tuning,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Kimi Team (2025) “Kimi K2: Open Agentic Intelligence,” *arXiv preprint arXiv:2507.20534* [Preprint].

Kimi Team (2026) “Kimi K3: Open Frontier Intelligence,” *arXiv preprint arXiv:2607.24653* [Preprint].

Kirk, R. *et al.* (2024) “Understanding the Effects of RLHF on LLM Generalisation and Diversity,” in *International Conference on Learning Representations*.

Ko, J. *et al.* (2025) "DistiLLM-2: A Contrastive Approach Boosts the Distillation of LLMs," in *International Conference on Machine Learning*.

Ko, J. *et al.* (2024) "DistiLLM: Towards Streamlined Distillation for Large Language Models," in *International Conference on Machine Learning*.

Korbak, T., Perez, E. and Buckley, C.L. (2022) "RL with KL Penalties is Better Viewed as Bayesian Inference," in *Findings of the Association for Computational Linguistics: EMNLP 2022*.

Lambert, N. *et al.* (2024) "Tülu 3: Pushing Frontiers in Open Language Model Post-Training," *arXiv preprint arXiv:2411.15124* [Preprint].

Le, T.A. (2017) *Reverse vs Forward KL*. Available at: https://www.tuananhle.co.uk/notes/reverse-forward-kl.html.

Li, C.T. and Farnia, F. (2023) "Mode-Seeking Divergences: Theory and Applications to GANs," in *International Conference on Artificial Intelligence and Statistics*. (PMLR), pp. 8321–8350.

Li, Y. *et al.* (2026) "Rethinking On-Policy Distillation of Large Language Models: Phenomenology, Mechanism, and Recipe," *arXiv preprint arXiv:2604.13016* [Preprint].

Lin, A. *et al.* (2020) "Autoregressive Knowledge Distillation through Imitation Learning," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.

Liu, K. *et al.* (2025b) "Rethinking KL Regularization in RLHF: From Value Estimation to Gradient Optimization," *arXiv preprint arXiv:2510.01555* [Preprint].

Liu, M. *et al.* (2025a) "ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models," *arXiv preprint arXiv:2505.24864* [Preprint].

Liu, Y. *et al.* (2026) "Your Teacher Can't Help You Here: Combating Supervision Fidelity Decay in On-Policy Distillation," *arXiv preprint arXiv:2605.30833* [Preprint].

Liu, Z. *et al.* (2025) "Understanding R1-Zero-Like Training: A Critical Perspective," *arXiv preprint arXiv:2503.20783* [Preprint].

Lu, K. and Thinking Machines Lab (2025) *On-Policy Distillation*. Available at: https://thinkingmachines.ai/blog/on-policy-distillation/.

Luong, Tran and Chen (2026) "Diversity-Aware Reverse KL Divergence for LLM Distillation," *arXiv preprint arXiv:2604.00223* [Preprint].

MacKay, D.J.C. (2003) *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press.

Malinin, A. and Gales, M. (2019) "Reverse KL-Divergence Training of Prior Networks: Improved Uncertainty and Adversarial Robustness," in *Advances in Neural Information Processing Systems*.

Meng, Y., Xia, M. and Chen, D. (2024) "SimPO: Simple Preference Optimization with a Reference-Free Reward," in *Advances in Neural Information Processing Systems*.

Meta AI (2024) *Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models*. Available at: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/.

Meta AI (2025) *The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation*. Available at: https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

Minka, T. (2005) *Divergence Measures and Message Passing*. technical report MSR-TR-2005-173.

Mistral AI (2025) "Magistral," *arXiv preprint arXiv:2506.10910* [Preprint].

Muralidharan, S. *et al.* (2024) "Compact Language Models via Pruning and Knowledge Distillation," *arXiv preprint arXiv:2407.14679* [Preprint].

Murphy, K.P. (2022) *Probabilistic Machine Learning: An Introduction*. MIT Press.

Murphy, K.P. (2023) *Probabilistic Machine Learning: Advanced Topics*. MIT Press.

Nowozin, S., Cseke, B. and Tomioka, R. (2016) "f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization," in *Advances in Neural Information Processing Systems*.

NVIDIA (2025a) "Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models," *arXiv preprint arXiv:2504.03624* [Preprint].

NVIDIA (2025b) "NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model," *arXiv preprint arXiv:2508.14444* [Preprint].

NVIDIA (2026b) "Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning," *arXiv preprint arXiv:2606.15007* [Preprint].

NVIDIA (2026a) "Nemotron-Cascade 2: Post-Training LLMs with Cascade RL and Multi-Domain On-Policy Distillation," *arXiv preprint arXiv:2603.19220* [Preprint].

OLMo Team (2025) "2 OLMo 2 Furious," *arXiv preprint arXiv:2501.00656* [Preprint].

OpenAI (2023) "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774* [Preprint].

Ouyang, L. *et al.* (2022) "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems*.

Qwen Team (2025) "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388* [Preprint].

Rafailov, R. *et al.* (2023) "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," in *Advances in Neural Information Processing Systems*.

Schulman, J. (2020) *Approximating KL Divergence*. Available at: http://joschu.net/blog/kl-approx.html.

Sessa, P.G. *et al.* (2024) "BOND: Aligning LLMs with Best-of-N Distillation," *arXiv preprint arXiv:2407.14622* [Preprint].

Shao, Z. *et al.* (2024) "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300* [Preprint].

Shen, Z. *et al.* (2026) "On the Geometry of On-Policy Distillation," *arXiv preprint arXiv:2606.07082* [Preprint].

Soletskyi, R., Gabrié, M. and Loureiro, B. (2024) "A Theoretical Perspective on Mode Collapse in Variational Inference," *arXiv preprint arXiv:2410.13300* [Preprint].

Song, M., Zheng, M. and others (2026) "A Survey of On-Policy Distillation for Large Language Models," *arXiv preprint arXiv:2604.00626* [Preprint].

Stiennon, N. *et al.* (2020) "Learning to Summarize from Human Feedback," in *Advances in Neural Information Processing Systems*.

Tang, Y. and Munos, R. (2025) "On a Few Pitfalls in KL Divergence Gradient Estimation for RL," *arXiv preprint arXiv:2506.09477* [Preprint].

Theis, L., Oord, A. van den and Bethge, M. (2016) "A Note on the Evaluation of Generative Models," in *International Conference on Learning Representations*.

Turner, R.E. and Sahani, M. (2011) "Two Problems with Variational Expectation Maximisation for Time-Series Models," in D. Barber, A.T. Cemgil, and S. Chiappa (eds.) *Bayesian Time Series Models*. Cambridge University Press, pp. 104–124.

Wang, C. *et al.* (2024) "Beyond Reverse KL: Generalizing Direct Preference Optimization with Diverse Divergence Constraints," in *International Conference on Learning Representations*.

Wang, G. *et al.* (2025) "ABKD: Pursuing a Proper Allocation of the Probability Mass in Knowledge Distillation via Alpha-Beta-Divergence," in *International Conference on Machine Learning*.

Wen, Y. *et al.* (2023) "f-Divergence Minimization for Sequence-Level Knowledge Distillation," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

Wu *et al.* (2026) "f-Divergence Regularized RLHF: Two Tales of Sampling and Unified Analyses," *arXiv preprint arXiv:2605.06977* [Preprint].

Wu, T. *et al.* (2025) "Rethinking Kullback-Leibler Divergence in Knowledge Distillation for Large Language Models," in *Proceedings of the 31st International Conference on Computational Linguistics*.

Xiaomi LLM-Core Team (2026) "MiMo-V2-Flash Technical Report," *arXiv preprint arXiv:2601.02780* [Preprint].

Xin, H. *et al.* (2026) “Escaping the KL Agreement Trap in On-Policy Distillation,” *arXiv preprint arXiv:2606.09471* [Preprint].

Xu, W. *et al.* (2025) “Speculative Knowledge Distillation: Bridging the Teacher-Student Gap Through Interleaved Sampling,” in *International Conference on Learning Representations*.

Yao, J. *et al.* (2026) “Future-KL Regularized GRPO: Process-Level Credit Assignment from f-Divergence Regularization,” *arXiv preprint arXiv:2601.10201* [Preprint].

Yu, Q. *et al.* (2025) “DAPO: An Open-Source LLM Reinforcement Learning System at Scale,” *arXiv preprint arXiv:2503.14476* [Preprint].

Yu, Z. *et al.* (2026) “Mismatch Matters: On-Policy Distillation Beyond Token Agreement,” *arXiv preprint arXiv:2608.09836* [Preprint].

Zhang, Y. *et al.* (2025) “On the Design of KL-Regularized Policy Gradient Algorithms for LLM Reasoning,” *arXiv preprint arXiv:2505.17508* [Preprint].

Zhao, H. *et al.* (2026) “OPD+: Rethinking the Advantage Design for On-Policy Distillation,” *arXiv preprint arXiv:2606.01039* [Preprint].

Zheng, B. *et al.* (2026) “SCOPE: Signal-Calibrated On-Policy Distillation Enhancement with Dual-Path Adaptive Weighting,” *arXiv preprint arXiv:2604.10688* [Preprint].

Zheng, C. *et al.* (2025) “Group Sequence Policy Optimization,” *arXiv preprint arXiv:2507.18071* [Preprint].

Ziegler, D.M. *et al.* (2019) “Fine-Tuning Language Models from Human Preferences,” *arXiv preprint arXiv:1909.08593* [Preprint].